

Cuaderno de Algoritmos Cuánticos

Deutsch, Deutsch–Jozsa, Simon y Grover

Notas de estudio sobre el capítulo 6 (secciones 6.1–6.4) de
N. S. Yanofsky y M. A. Mannucci, *Quantum Computing for Computer Scientists*

Elia Torres Simón

Trabajo de Fin de Grado — Grado en Matemáticas, UCM

4 de octubre de 2026

Sobre este cuaderno

Este cuaderno desarrolla la lectura asignada para la próxima reunión: las secciones 6.1 a 6.4 del capítulo 6 (*Algorithms*) de Yanofsky y Mannucci [1]. **La sección 6.5, el algoritmo de factorización de Shor, queda fuera.** Cada algoritmo se explica con todos los pasos intermedios, se demuestran los resultados que el libro solo enuncia (por ejemplo, el número de iteraciones de Grover, que el libro declara “fuera del alcance del texto”), se resuelven los ejercicios del libro y se comprueba todo numéricamente en Qiskit.

Cada capítulo tiene la misma estructura que el *Cuaderno de Computación Cuántica*:

- **El problema y la solución clásica**, que hace de punto de referencia (el papel que allí tenía la “información clásica”).
- **El algoritmo cuántico**, con la teoría completa, las demostraciones y los ejercicios resueltos.
- **Implementación en Qiskit**, donde se comprueban numéricamente los resultados.
- **Resumen: un ejemplo guiado**, que recorre todos los conceptos con un solo ejemplo y números concretos.

Capítulo	Yanofsky	Páginas del libro
1. El algoritmo de Deutsch	Sección 6.1	170–179
2. El algoritmo de Deutsch–Jozsa	Sección 6.2	179–187
3. El algoritmo de periodicidad de Simon	Sección 6.3	187–195
4. El algoritmo de búsqueda de Grover	Sección 6.4	195–204
(<i>excluido</i>) El algoritmo de Shor	Sección 6.5	204–219

Las referencias del tipo [Y 6.27] remiten a la ecuación (6.27) del libro, para poder leer los dos textos en paralelo. Los ejercicios del libro aparecen resueltos con su numeración original (*Ejercicio 6.1.3*, etc.).

Código de colores de las cajas:

- Rosa : definiciones, proposiciones y teoremas.
- Gris : demostraciones y pasos de los ejemplos guiados.
- Salmón : notas aclaratorias e intuición.
- Salmón intenso : **diferencias de convención con Yanofsky** (léelas antes de comparar una fórmula con el libro).
- Rosa claro : ejemplos con números y ejercicios resueltos del libro.

Las cajas de **Conexión con el TFG** señalan las ideas que reaparecerán en los kernels cuánticos.

El código completo está en `algoritmos_yanofsky.py`, ejecutado con Qiskit **versión 2.2.3**; las figuras están en la carpeta `figuras_yanofsky`.

Índice general

Antes de empezar: convenciones	7
1. El algoritmo de Deutsch	11
1.1. El problema y la solución clásica	13
1.1.1. Las funciones como matrices	13
1.2. El algoritmo cuántico	15
1.2.1. El oráculo U_f	15
1.2.2. Primer intento: evaluar sobre una superposición	16
1.2.3. Segundo intento: la salida en $ -\rangle$ y el retroceso de fase	17
1.2.4. El algoritmo de Deutsch	18
1.3. Implementación en Qiskit	21
1.3.1. Experimento 1: los cuatro oráculos con puertas	21
1.3.2. Experimento 2: el algoritmo	21
1.4. Resumen: un ejemplo guiado	23
2. El algoritmo de Deutsch–Jozsa	25
2.1. El problema y la solución clásica	27
2.2. El algoritmo cuántico	29
2.2.1. El oráculo de n qubits	29
2.2.2. La transformada de Hadamard de n qubits	30
2.2.3. Primer intento	32
2.2.4. El algoritmo	33
2.3. Implementación en Qiskit	37
2.3.1. Un oráculo para cualquier función	37
2.3.2. Experimento 2: Deutsch–Jozsa con funciones de 2 bits	37
2.3.3. Experimento 3: una función equilibrada de 3 bits al azar	38
2.4. Resumen: un ejemplo guiado	39
3. El algoritmo de periodicidad de Simon	41
3.1. El problema y la solución clásica	43
3.2. El algoritmo cuántico	45
3.2.1. El circuito	45
3.2.2. La parte clásica: ecuaciones lineales sobre $\mathbb{F}_2$	48
3.2.3. ¿Cuántas ejecuciones hacen falta?	49
3.3. Implementación en Qiskit	53
3.3.1. Experimento 4: el algoritmo de Simon con las funciones del libro	53
3.3.2. Experimento 5: el algoritmo completo, ejecución a ejecución	54
3.4. Resumen: un ejemplo guiado	55

4. El algoritmo de búsqueda de Grover	57
4.1. El problema y la solución clásica	59
4.2. El algoritmo cuántico	61
4.2.1. El oráculo y un primer intento	61
4.2.2. Primer ingrediente: la inversión de fase	61
4.2.3. Segundo ingrediente: la inversión respecto de la media	62
4.2.4. El algoritmo	63
4.2.5. Demostración: por qué $\frac{\pi}{4}\sqrt{N}$ iteraciones	65
4.3. Implementación en Qiskit	69
4.3.1. Experimento 6: el oráculo y la difusión con puertas	69
4.3.2. Experimento 7: el algoritmo completo	70
4.4. Resumen: un ejemplo guiado	73

Antes de empezar: convenciones

Yanofsky y Mannucci escriben los estados de varios qubits con una convención distinta de la de Qiskit (y de la del *Cuaderno de Computación Cuántica*). La diferencia no cambia ningún resultado físico, pero sí cambia el aspecto de las matrices y el orden de los factores en los productos tensoriales. Este cuaderno usa **siempre la convención de Qiskit**; aquí se explica la traducción una sola vez, y en cada capítulo una caja salmón recuerda dónde aparece.

El orden de los qubits

Convención de este cuaderno (la de Qiskit)

El cable **de arriba** del circuito es el qubit q_0 y corresponde al factor **de la derecha** del producto tensorial y al último carácter de las cadenas de bits. Con n qubits,

$$|b_{n-1} \cdots b_1 b_0\rangle = |b_{n-1}\rangle \otimes \cdots \otimes |b_1\rangle \otimes |b_0\rangle, \quad b_0 = \text{valor de } q_0 \text{ (cable de arriba)}.$$

Leer los cables de abajo arriba equivale a leer el ket de izquierda a derecha.

Yanofsky hace lo contrario: en sus circuitos el cable de arriba es el factor de la **izquierda**. Por ejemplo, en el algoritmo de Deutsch el cable de arriba lleva el qubit de consulta x y el de abajo el qubit de salida y , y el libro escribe el estado conjunto como $|x, y\rangle = |x\rangle \otimes |y\rangle$.

En este cuaderno **los circuitos se dibujan igual que en el libro** (consulta arriba, salida abajo), que es además como los dibuja Qiskit, pero los estados se escriben en el orden de Qiskit:

$$\underbrace{|x, y\rangle}_{\text{Yanofsky}} \longleftrightarrow \underbrace{|y\rangle |x\rangle}_{\text{este cuaderno}} = |y\rangle \otimes |x\rangle.$$

Objeto	Yanofsky	Este cuaderno (Qiskit)
Estado de entrada de Deutsch	$ 0, 1\rangle = 0\rangle \otimes 1\rangle$	$ 1\rangle 0\rangle = 10\rangle$
Oráculo	$U_f x, y\rangle = x, y \oplus f(x)\rangle$	$U_f y\rangle x\rangle = y \oplus f(x)\rangle x\rangle$
Hadamard solo en el cable de arriba	$H \otimes I$	$I \otimes H$
Hadamard solo en el cable de abajo	$I \otimes H$	$H \otimes I$
Algoritmo de Deutsch completo	$(H \otimes I) U_f (H \otimes H) 0, 1\rangle$	$(I \otimes H) U_f (H \otimes H) 1\rangle 0\rangle$
Índice de una entrada de U_f	fila/columna xy	fila/columna yx

Proposición: las dos matrices de U_f son conjugadas por el SWAP

Sea U^Y la matriz de un operador de dos qubits escrita en el orden de Yanofsky (índices xy) y U^Q la del mismo operador en el orden de Qiskit (índices yx). Entonces

$$U^Q = \text{SWAP } U^Y \text{ SWAP}, \quad \text{SWAP}(|a\rangle \otimes |b\rangle) = |b\rangle \otimes |a\rangle.$$

En particular, U^Q y U^Y tienen los mismos autovalores, la misma traza y son unitarias a la vez: son **la misma operación escrita en otra base ordenada**.

Demostración

Las dos matrices representan la misma aplicación lineal; solo cambia el orden en que se listan los vectores de la base. La entrada de U^Y en la fila $x'y'$ y la columna xy es el coeficiente de $|x'\rangle \otimes |y'\rangle$ en la imagen de $|x\rangle \otimes |y\rangle$; la entrada de U^Q en la fila $y'x'$ y la columna yx es el mismo número. Reordenar filas y columnas según la permutación $xy \mapsto yx$ es exactamente multiplicar a izquierda y derecha por la matriz de permutación de esa biyección, que es el SWAP (y $\text{SWAP}^{-1} = \text{SWAP}$). La conjugación por una matriz invertible conserva autovalores y traza, y la conjugación por una unitaria conserva la unitariedad. $\square$

Nota Aclaratoria: los nombres de los bits en las cadenas

Yanofsky nombra los bits de una cadena de izquierda a derecha, $\mathbf{x} = x_0x_1 \cdots x_{n-1}$ (y en algún ejemplo $\mathbf{c} = c_1c_2c_3$). En Qiskit el último carácter es $\mathbf{q}_0$. Para no arrastrar dos numeraciones, en este cuaderno **las cadenas se usan como etiquetas**: la función que el libro define sobre la cadena 10 se define aquí sobre la misma cadena 10, leída al modo de Qiskit ($\mathbf{q}_1 = 1$, $\mathbf{q}_0 = 0$). Todas las cuentas del libro siguen valiendo tal cual, porque ningún algoritmo depende del nombre que se dé a cada posición: el producto binario $\mathbf{x} \cdot \mathbf{z}$ de la siguiente sección es simétrico respecto de cualquier reordenación aplicada a la vez a las dos cadenas. Cuando haga falta hablar de una posición concreta se dirá “el primer bit por la izquierda”.

Producto interno y producto binario

Producto interno de estados. Como en el cuaderno, $\langle \psi | \phi \rangle = \sum_a \overline{\psi_a} \phi_a$ es **antilineal en el primer argumento**. Yanofsky usa la misma convención ($\langle V_1, V_2 \rangle = V_1^\dagger V_2$, capítulo 2 del libro), así que aquí no hay que traducir nada.

Producto binario de cadenas. A partir de la sección 6.2, Yanofsky usa $\langle \mathbf{x}, \mathbf{y} \rangle$ para algo distinto: la paridad del número de posiciones en que dos cadenas de bits valen 1 a la vez. Para no confundirlo con el producto interno de estados, aquí se escribe con un punto:

$$\mathbf{x} \cdot \mathbf{z} = x_{n-1}z_{n-1} \oplus \cdots \oplus x_1z_1 \oplus x_0z_0 \in \{0, 1\} \quad (\text{Yanofsky: } \langle \mathbf{x}, \mathbf{z} \rangle).$$

Por ejemplo, $101 \cdot 111 = 1 \oplus 0 \oplus 1 = 0$ y $110 \cdot 011 = 0 \oplus 1 \oplus 0 = 1$.

El esquema común de los algoritmos

Yanofsky abre el capítulo con el esquema que siguen todos los algoritmos cuánticos que se van a estudiar [Y p. 171]:

1. Los qubits empiezan en un **estado clásico** (un vector de la base estándar).
2. Se ponen en una **superposición** de muchos estados (casi siempre con puertas de Hadamard).

3. Se aplican **operaciones unitarias** a esa superposición, entre ellas la que evalúa la función.
4. Se **miden** los qubits.

Definición: modelo de consultas

En los cuatro problemas de este cuaderno se da una función f como **caja negra** (un **oráculo**): se puede evaluar en las entradas que se quiera, pero no se puede “mirar dentro” para ver cómo está definida. El coste de un algoritmo se mide por el **número de consultas**, es decir, de evaluaciones de f . En el caso cuántico, una consulta es una aplicación de la puerta U_f , que se define en el Capítulo 1.

Esta forma de medir el coste es la que permite comparar con rigor: se cuentan consultas, no segundos, y se demuestra que ningún algoritmo clásico puede hacerlo con menos. Así se obtienen las ventajas de la tabla siguiente, que se irán demostrando capítulo a capítulo.

Problema	Clásico (determinista)	Cuántico	Ventaja
Deutsch ($n = 1$)	2 consultas	1 consulta	$2 \rightarrow 1$
Deutsch–Jozsa	$2^{n-1} + 1$	1	exponencial
Simon	$2^{n-1} + 1$	menos de $n + 1$ ejecuciones en media	exponencial
Grover	2^n (peor caso)	unas $\frac{\pi}{4}\sqrt{2^n}$	cuadrática

Capítulo 1

El algoritmo de Deutsch

El algoritmo de Deutsch [2, 4] es el algoritmo cuántico más sencillo. Resuelve un problema algo artificial (decidir si una función de un bit es constante o equilibrada), pero contiene ya los tres ingredientes de todos los algoritmos de este cuaderno: **evaluar una función sobre una superposición**, **devolver el resultado en la fase** (el “retroceso de fase”) e **interferir** con puertas de Hadamard para que la respuesta se pueda leer midiendo. Corresponde a la sección 6.1 de Yanofsky.

1.1. El problema y la solución clásica

Hay exactamente $2^2 = 4$ funciones $f : \{0, 1\} \rightarrow \{0, 1\}$. Con los nombres del *Cuaderno de Computación Cuántica* [Y 6.1]:

a	$f_1(a)$	a	$f_2(a)$	a	$f_3(a)$	a	$f_4(a)$
0	0	0	0	0	1	0	1
1	0	1	1	1	0	1	1

Definición: funciones constantes y equilibradas (un bit)

Una función $f : \{0, 1\} \rightarrow \{0, 1\}$ es **constante** si $f(0) = f(1)$ y **equilibrada** si $f(0) \neq f(1)$, es decir, si es biyectiva. De las cuatro, f_1 (constante 0) y f_4 (constante 1) son constantes, y f_2 (identidad) y f_3 (NOT) son equilibradas.

El problema de Deutsch

Se da $f : \{0, 1\} \rightarrow \{0, 1\}$ como caja negra. Hay que decidir si f es **constante** o **equilibrada** con el menor número posible de consultas.

Observemos que f es constante si y solo si $f(0) \oplus f(1) = 0$. El problema pide, por tanto, calcular **un solo bit**, $f(0) \oplus f(1)$, que depende de los dos valores de f .

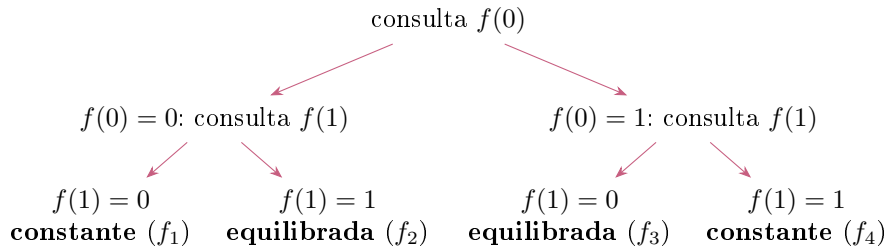
Proposición: un algoritmo clásico necesita dos consultas

Todo algoritmo clásico determinista que resuelva el problema de Deutsch sin error tiene que evaluar f en 0 y en 1.

Demostración

Supongamos que un algoritmo solo hace una consulta, y que pregunta por $f(0)$ (el caso $f(1)$ es simétrico). Sea $a = f(0)$ la respuesta. Hay dos funciones compatibles con esa información: la constante a y la equilibrada con $f(0) = a$, $f(1) = 1 - a$. Ambas producen exactamente la misma ejecución del algoritmo (la misma consulta y la misma respuesta), así que el algoritmo da la misma salida para las dos y se equivoca en una de ellas. Por tanto hacen falta dos consultas, y con dos basta: se calcula $f(0) \oplus f(1)$. $\square$

El árbol de decisión de Yanofsky [Y 6.2] expresa lo mismo: tras la primera consulta, cada rama sigue conteniendo una función constante y una equilibrada.



1.1.1. Las funciones como matrices

Como en el Capítulo 1 del cuaderno, cada función $f : \{0, 1\} \rightarrow \{0, 1\}$ actúa sobre los vectores de la base mediante la matriz $M_f = \sum_a |f(a)\rangle \langle a|$, cuya columna a es $|f(a)\rangle$ (la columna es la entrada y la fila es la salida, como dice Yanofsky en [Y 6.5]).

Ejercicio 6.1.1 (Yanofsky): las matrices de las cuatro funciones

La función NOT de [Y 6.4] tiene matriz M_3 (es la de [Y 6.5]). Las cuatro son

$$M_1 = \begin{pmatrix} 1 & 1 \\ 0 & 0 \end{pmatrix}, \quad M_2 = \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix}, \quad M_3 = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}, \quad M_4 = \begin{pmatrix} 0 & 0 \\ 1 & 1 \end{pmatrix}.$$

Por ejemplo, $M_4 |0\rangle = |1\rangle = |f_4(0)\rangle$ y $M_4 |1\rangle = |1\rangle = |f_4(1)\rangle$.

M_1 y M_4 tienen determinante 0: no son invertibles y, por tanto, **no son unitarias**. Un ordenador cuántico no puede aplicarlas tal cual. Hace falta otra forma de evaluar f , que es el punto de partida de la sección siguiente.

1.2. El algoritmo cuántico

1.2.1. El oráculo U_f

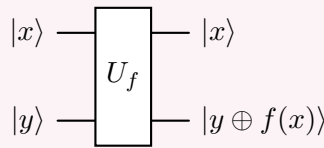
La solución estándar para evaluar una función no invertible de forma reversible es **conservar la entrada** y escribir la salida en un qubit adicional mediante un XOR [Y 6.6].

Definición: oráculo de una función

Para $f : \{0, 1\} \rightarrow \{0, 1\}$, el **oráculo** U_f es la operación de dos qubits definida sobre la base estándar por

$$U_f(|y\rangle |x\rangle) = |y \oplus f(x)\rangle |x\rangle, \quad x, y \in \{0, 1\},$$

y extendida por linealidad. El qubit x (cable de arriba, q_0) es la **entrada** o **consulta**; el qubit y (cable de abajo, q_1) es la **salida**.



Con $y = 0$ se obtiene $U_f |0\rangle |x\rangle = |f(x)\rangle |x\rangle$: el oráculo **calcula** $f(x)$.

Diferencia con Yanofsky: el orden de los factores en el oráculo

Yanofsky escribe $U_f |x, y\rangle = |x, y \oplus f(x)\rangle$, con la entrada a la izquierda. Es el mismo circuito; solo cambia el orden en el ket. En consecuencia, las matrices de U_f del libro y las de este cuaderno **no coinciden entrada a entrada**: son conjugadas por el SWAP (véase “Antes de empezar”).

Proposición: U_f es una permutación, es su propia inversa y es unitaria

Para toda f , la matriz de U_f es una matriz de permutación, $U_f^2 = I$ y $U_f^\dagger = U_f = U_f^{-1}$.

Demostración

Permutación. U_f envía cada vector de la base estándar a otro vector de la base estándar, $|y\rangle |x\rangle \mapsto |y \oplus f(x)\rangle |x\rangle$. La aplicación $(y, x) \mapsto (y \oplus f(x), x)$ es una biyección de $\{0, 1\}^2$, porque es su propia inversa (punto siguiente). Una matriz cuyas columnas son los vectores de la base estándar permutados es una matriz de permutación.

Involución. Aplicando U_f dos veces [Y 6.8]:

$$\begin{aligned} U_f^2 |y\rangle |x\rangle &= U_f |y \oplus f(x)\rangle |x\rangle = |(y \oplus f(x)) \oplus f(x)\rangle |x\rangle \\ &= |y \oplus (f(x) \oplus f(x))\rangle |x\rangle = |y \oplus 0\rangle |x\rangle = |y\rangle |x\rangle, \end{aligned}$$

usando la asociatividad de $\oplus$ y que $b \oplus b = 0$ para todo bit b (Yanofsky dice que $\oplus$ es “idempotente”, pero la propiedad que se usa es esta: **todo bit es su propio opuesto**). Como U_f^2 coincide con la identidad sobre una base, $U_f^2 = I$.

Unitariedad. Las matrices de permutación P son reales y cumplen $P^T P = I$ (las columnas son vectores distintos de la base canónica, luego ortonormales). Así $U_f^\dagger U_f = I$. Combinando con $U_f^2 = I$ se obtiene $U_f^\dagger = U_f^{-1} = U_f$. $\square$

La misma demostración vale sin cambios para los oráculos de varios qubits de los capítulos siguientes, en los que x e y son cadenas de bits y $\oplus$ es el XOR bit a bit.

Ejercicios 6.1.2 y 6.1.3 (Yanofsky): las cuatro matrices U_f

Ordenamos la base como $|y\rangle |x\rangle$: $|00\rangle, |01\rangle, |10\rangle, |11\rangle$. La columna yx es la imagen de $|y\rangle |x\rangle$:

$$U_{f_1} = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}, \quad U_{f_2} = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 \end{pmatrix},$$

$$U_{f_3} = \begin{pmatrix} 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 \\ 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}, \quad U_{f_4} = \begin{pmatrix} 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \\ 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \end{pmatrix}.$$

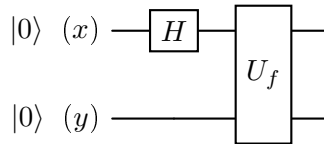
Comprobación de una columna de U_{f_3} : $U_{f_3} |0\rangle |0\rangle = |0 \oplus 1\rangle |0\rangle = |10\rangle$, que es el tercer vector de la base: la primera columna tiene su 1 en la tercera fila.

Las cuatro son simétricas y reales, luego **autoadjuntas** ($U^\dagger = \overline{U}^T = U$), y por la proposición anterior $U^2 = I$; por tanto $U^\dagger U = I$: son unitarias. En forma de circuito, $U_{f_1} = I$ (no hace nada), U_{f_2} es un CNOT con control en x y objetivo en y , U_{f_3} es ese CNOT seguido de una X sobre y , y $U_{f_4} = X \otimes I$ (una X sobre y).

Comparación con el libro. La matriz de U_{f_3} en el orden de Yanofsky [Y 6.9] es $\begin{pmatrix} 0 & 1 & 0 & 0 \\ 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$ (índices xy): intercambia $|0,0\rangle \leftrightarrow |0,1\rangle$. La nuestra intercambia $|00\rangle \leftrightarrow |10\rangle$, que en el orden yx son los mismos dos estados ($x = 0$, $y = 0$ o 1). El Ejercicio 6.1.2 pide su adjunta: como es real y simétrica, es ella misma, y es su propia inversa.

1.2.2. Primer intento: evaluar sobre una superposición

La primera idea [Y 6.12] es poner la entrada en $|+\rangle = H|0\rangle = \frac{1}{\sqrt{2}}(|0\rangle + |1\rangle)$ y la salida en $|0\rangle$, para evaluar f “en 0 y en 1 a la vez”:



Los estados son $|\varphi_0\rangle = |0\rangle |0\rangle$, $|\varphi_1\rangle = (I \otimes H) |0\rangle |0\rangle = \frac{1}{\sqrt{2}}(|0\rangle |0\rangle + |0\rangle |1\rangle)$ y, por linealidad,

$$|\varphi_2\rangle = U_f |\varphi_1\rangle = \frac{1}{\sqrt{2}}(|f(0)\rangle |0\rangle + |f(1)\rangle |1\rangle). \quad [\text{Y 6.18}]$$

Ejercicio 6.1.4 (Yanofsky): $|\varphi_2\rangle$ del primer intento para las cuatro funciones

f	$ \varphi_2\rangle$	¿qué se ve al medir?
f_1	$\frac{1}{\sqrt{2}}(00\rangle + 01\rangle) = 0\rangle +\rangle$	$y = 0$ siempre; x al azar
f_2	$\frac{1}{\sqrt{2}}(00\rangle + 11\rangle)$	$y = x$, cada par con prob. $\frac{1}{2}$
f_3	$\frac{1}{\sqrt{2}}(10\rangle + 01\rangle)$	$y \neq x$, cada par con prob. $\frac{1}{2}$
f_4	$\frac{1}{\sqrt{2}}(10\rangle + 11\rangle) = 1\rangle +\rangle$	$y = 1$ siempre; x al azar

El caso f_3 es el de [Y 6.19]: Yanofsky obtiene $\frac{1}{\sqrt{2}}(|0, 1\rangle + |1, 0\rangle)$, que con el orden cambiado es $\frac{1}{\sqrt{2}}(|10\rangle + |01\rangle)$, el mismo vector.

Nota Aclaratoria: por qué el primer intento no sirve

Medir $|\varphi_2\rangle$ da **un** par (y, x) con $y = f(x)$ y x al azar: exactamente lo mismo que una consulta clásica a una entrada elegida al azar. Por ejemplo, el resultado $(y, x) = (0, 0)$ es posible tanto para f_1 como para f_2 , así que una medición nunca permite decidir con certeza. La superposición “evalúa f en las dos entradas”, pero al medir solo se recupera una de ellas. **La superposición sola no basta: hace falta interferencia.**

Obsérvese también que para las funciones equilibradas $|\varphi_2\rangle$ es un **estado de Bell**, entrelazado (Capítulo 2 del cuaderno): $\frac{1}{\sqrt{2}}(|00\rangle + |11\rangle) = |\phi^+\rangle$ y $\frac{1}{\sqrt{2}}(|01\rangle + |10\rangle) = |\psi^+\rangle$.

1.2.3. Segundo intento: la salida en $|-\rangle$ y el retroceso de fase

La segunda idea [Y 6.21] es poner el qubit de salida en $|-\rangle = H|1\rangle = \frac{1}{\sqrt{2}}(|0\rangle - |1\rangle)$. El resultado es la observación más importante del capítulo.

Lema: retroceso de fase (*phase kickback*)

Para todo $x \in \{0, 1\}$,

$$U_f(|-\rangle|x\rangle) = (-1)^{f(x)}|-\rangle|x\rangle. \quad [\text{Y 6.27}]$$

Es decir, $|-\rangle|x\rangle$ es un **autovector** de U_f con autovalor $(-1)^{f(x)}$: el valor de f no se escribe en el qubit de salida, sino que “retrocede” en forma de **signo** sobre la entrada.

Demostración

Por linealidad,

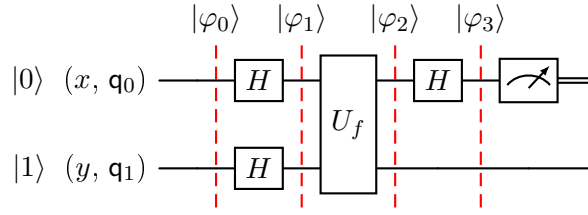
$$U_f(|-\rangle|x\rangle) = \frac{1}{\sqrt{2}}(U_f|0\rangle|x\rangle - U_f|1\rangle|x\rangle) = \frac{1}{\sqrt{2}}(|f(x)\rangle|x\rangle - |1 \oplus f(x)\rangle|x\rangle).$$

Si $f(x) = 0$, esto es $\frac{1}{\sqrt{2}}(|0\rangle - |1\rangle)|x\rangle = |-\rangle|x\rangle$. Si $f(x) = 1$, es $\frac{1}{\sqrt{2}}(|1\rangle - |0\rangle)|x\rangle = -|-\rangle|x\rangle$. En ambos casos queda $(-1)^{f(x)}|-\rangle|x\rangle$. $\square$

Si x es un estado de la base, el signo es una **fase global** y no se observa (Capítulo 3 del cuaderno): por eso el segundo intento, con x fijo, tampoco da información [Y p. 176]. Pero si x está en superposición, cada componente recibe su **propio** signo y el signo pasa a ser una **fase relativa**, que sí se puede detectar.

1.2.4. El algoritmo de Deutsch

El algoritmo combina los dos intentos: la entrada en superposición y la salida en $|-\rangle$, y una Hadamard final sobre la entrada [Y 6.28].



En matrices, con el orden de Qiskit,

$$|\varphi_3\rangle = (I \otimes H) U_f (H \otimes H) |1\rangle |0\rangle .$$

Diferencia con Yanofsky: la fórmula del algoritmo

Yanofsky escribe [Y 6.29] $(H \otimes I) U_f (H \otimes H) |0, 1\rangle$. Su $H \otimes I$ actúa sobre el factor de la izquierda, que para él es el cable de arriba; aquí el cable de arriba es el factor de la derecha y la misma puerta es $I \otimes H$. Igualmente, su $|0, 1\rangle$ (arriba 0, abajo 1) es nuestro $|1\rangle |0\rangle$.

Teorema: corrección del algoritmo de Deutsch

Para toda $f : \{0, 1\} \rightarrow \{0, 1\}$,

$$|\varphi_3\rangle = (-1)^{f(0)} |-\rangle \otimes |f(0) \oplus f(1)\rangle .$$

En consecuencia, al medir el qubit de arriba se obtiene $f(0) \oplus f(1)$ **con probabilidad 1**: el resultado es 0 si f es constante y 1 si es equilibrada. El algoritmo usa **una sola** consulta.

Demostración

Seguimos el estado paso a paso.

Paso 0. $|\varphi_0\rangle = |1\rangle|0\rangle$.

Paso 1. $|\varphi_1\rangle = (H \otimes H)|1\rangle|0\rangle = H|1\rangle \otimes H|0\rangle = |-\rangle|+\rangle = |-\rangle \otimes \frac{1}{\sqrt{2}}(|0\rangle + |1\rangle)$.

Desarrollando, $|\varphi_1\rangle = \frac{1}{2}(|00\rangle + |01\rangle - |10\rangle - |11\rangle)$, que es [Y 6.32] con el orden cambiado.

Paso 2. Por linealidad y el lema del retroceso de fase,

$$|\varphi_2\rangle = \frac{1}{\sqrt{2}}(U_f|-\rangle|0\rangle + U_f|-\rangle|1\rangle) = |-\rangle \otimes \frac{1}{\sqrt{2}}((-1)^{f(0)}|0\rangle + (-1)^{f(1)}|1\rangle). \quad [\text{Y 6.34}]$$

Sacando factor común $(-1)^{f(0)}$ y usando $(-1)^{f(1)-f(0)} = (-1)^{f(0)\oplus f(1)}$ (para bits, $a-b$ y $a\oplus b$ tienen la misma paridad):

$$|\varphi_2\rangle = (-1)^{f(0)}|-\rangle \otimes \frac{1}{\sqrt{2}}(|0\rangle + (-1)^{f(0)\oplus f(1)}|1\rangle).$$

El paréntesis es $|+\rangle$ si $f(0)\oplus f(1)=0$ y $|-\rangle$ si $f(0)\oplus f(1)=1$.

Paso 3. Como $H|+\rangle = |0\rangle$ y $H|-\rangle = |1\rangle$, aplicar H al qubit de arriba da

$$|\varphi_3\rangle = (-1)^{f(0)}|-\rangle \otimes |f(0)\oplus f(1)\rangle.$$

Medición. El estado es un producto cuyo factor de arriba es un vector de la base, $|b\rangle$ con $b = f(0)\oplus f(1)$; por la regla de Born, el resultado es b con probabilidad $|(-1)^{f(0)}|^2 = 1$.

□

Ejercicios 6.1.5 y 6.1.6 (Yanofsky): $|\varphi_2\rangle$ y $|\varphi_3\rangle$ para las cuatro funciones

En todos los casos el qubit de abajo es $|-\rangle$; la tabla da el factor del qubit de arriba.

f	$f(0), f(1)$	arriba en $ \varphi_2\rangle$	arriba en $ \varphi_3\rangle$	resultado
f_1	0, 0	$+ +\rangle$	$+ 0\rangle$	0 (constante)
f_2	0, 1	$+ -\rangle$	$+ 1\rangle$	1 (equilibrada)
f_3	1, 0	$- -\rangle$	$- 1\rangle$	1 (equilibrada)
f_4	1, 1	$- +\rangle$	$- 0\rangle$	0 (constante)

Para f_3 (el ejemplo del libro, [Y 6.35] y [Y 6.41]): $|\varphi_2\rangle = |-\rangle \otimes \frac{1}{\sqrt{2}}(-|0\rangle + |1\rangle) = -|-\rangle|-\rangle$ y $|\varphi_3\rangle = -|-\rangle|1\rangle$. Desarrollado en la base, $|\varphi_3\rangle = \frac{1}{\sqrt{2}}(-|01\rangle + |11\rangle)$.

Nota Aclaratoria: ¿hemos sacado información de la nada?

No [Y p. 178]. Para identificar cuál de las cuatro funciones es f hacen falta dos bits, y clásicamente cada consulta da un bit. El algoritmo cuántico **no** identifica f : la fase global ± 1 distingue f_1 de f_4 y f_2 de f_3 , pero ninguna medición la detecta. Lo que hace es responder con una sola consulta a otra pregunta, “¿cuánto vale $f(0)\oplus f(1)$?”, que clásicamente necesitaba dos. Es lo que Yanofsky llama **cambiar la pregunta** (cambio de base): las Hadamard pasan a la base $\{|+\rangle, |-\rangle\}$, en la que la información útil está en las fases, y vuelven a la base estándar para leerla [Y 6.43].

Una forma de verlo: U_{f_2} es un CNOT con control en x y objetivo en y . En la base de Hadamard los papeles se invierten, $(H \otimes H)\text{CNOT}_{x\rightarrow y}(H \otimes H) = \text{CNOT}_{y\rightarrow x}$, así que, con y en $|-\rangle$ (que es $|1\rangle$ en la base de Hadamard), el “control” y actúa sobre x .

Una corrección al libro: el algoritmo no usa entrelazamiento

Yanofsky atribuye el funcionamiento del algoritmo a que “los qubits de arriba y de abajo están entrelazados” [Y p. 178]. No es así: como muestra la demostración, **en todos los pasos el estado es un producto** $|-\rangle \otimes (\dots)$. El mecanismo es el **retroceso de fase** (el qubit de salida es un autovector de U_f) seguido de **interferencia**. El entrelazamiento aparece en el *primer intento* (estados de Bell para f_2 y f_3), que es precisamente el que no funciona. Esta presentación con retroceso de fase es la de Cleve, Ekert, Macchiavello y Mosca [4].

Conexión con el TFG

El patrón “Hadamard, fases que dependen de la entrada, Hadamard y medir” es el mismo del mapa de características de Havlíček et al. [11]: allí las fases no son $(-1)^{f(x)}$, sino $e^{i\phi(\mathbf{x})}$ con ϕ dependiente del dato. En el Capítulo 2 se verá que la probabilidad de medir $|0 \dots 0\rangle$ es el módulo al cuadrado de un producto interno, igual que el kernel cuántico.

1.3. Implementación en Qiskit

Comprobamos el algoritmo con las cuatro funciones. El código completo está en `algoritmos_yanofsky.py`. Para ejecutar circuitos con mediciones usamos la misma función auxiliar que en el cuaderno, con una semilla para que los resultados sean reproducibles:

```
sampler = StatevectorSampler(seed=np.random.default_rng(2026))

def contar(circuito, shots=1000):
    """Ejecuta un circuito con mediciones y devuelve el recuento."""
    resultado = sampler.run([circuito], shots=shots).result()[0]
    return resultado.data.c.get_counts()
```

El nombre `c` es el que Qiskit da al registro clásico cuando el circuito se crea con `QuantumCircuit(n, m)`.

1.3.1. Experimento 1: los cuatro oráculos con puertas

Construimos cada U_f con las puertas del Ejercicio 6.1.3: q_0 es x y q_1 es y .

```
def oraculo_deutsch(nombre):
    qc = QuantumCircuit(2, name="U_f")
    if nombre == "f2":          # identidad: y <- y xor x
        qc.cx(0, 1)
    elif nombre == "f3":        # NOT: y <- y xor x xor 1
        qc.cx(0, 1)
        qc.x(1)
    elif nombre == "f4":        # constante 1: y <- y xor 1
        qc.x(1)
    return qc                   # f1 (constante 0): no hace nada
```

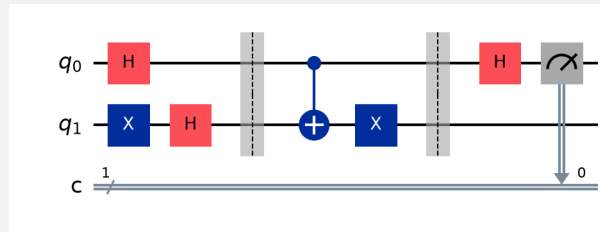
Para comprobar que son los oráculos correctos, los comparamos con la matriz de la definición, $U_f |y\rangle |x\rangle = |y \oplus f(x)\rangle |x\rangle$, que construye la función general `oraculo(f, n, m)` del programa (una `UnitaryGate` a partir de la matriz de permutación). La salida es `True` para las cuatro funciones.

Nota Aclaratoria: `cx(0, 1)` y el orden de Qiskit

`qc.cx(0, 1)` tiene el control en q_0 (x , arriba) y el objetivo en q_1 (y , abajo): hace $|y\rangle |x\rangle \mapsto |y \oplus x\rangle |x\rangle$, que es U_{f_2} . Su matriz en la base $|00\rangle, |01\rangle, |10\rangle, |11\rangle$ intercambia $|01\rangle \leftrightarrow |11\rangle$, como la U_{f_2} del Ejercicio 6.1.3. En el Capítulo 3 del cuaderno este CNOT se llamaba $\text{CNOT}_{Y \rightarrow X}$ cuando el control estaba en el cable de arriba: aquí el cable de arriba es x .

1.3.2. Experimento 2: el algoritmo

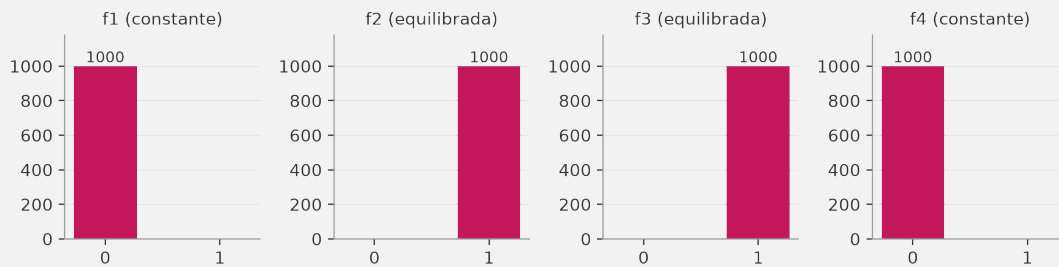
```
def deutsch(nombre):
    qc = QuantumCircuit(2, 1)
    qc.x(1)                # y = |1>
    qc.h([0, 1])
    qc.barrier()
    qc.compose(oraculo_deutsch(nombre), inplace=True)
    qc.barrier()
    qc.h(0)
    qc.measure(0, 0)
    return qc
```



El circuito de Deutsch para f_3 dibujado por Qiskit. Entre las dos barreras está el oráculo: un CNOT (x controla y) y una X sobre y . El cable de arriba es $q_0 = x$.

Con 1000 disparos para cada función se obtiene:

```
f1 {'0': 1000}
f2 {'1': 1000}
f3 {'1': 1000}
f4 {'0': 1000}
```



Resultados del algoritmo de Deutsch con 1000 disparos. Las funciones constantes dan siempre 0 y las equilibradas siempre 1, sin un solo error: el algoritmo es **determinista**, no probabilístico.

Por último, comprobamos el estado final exacto para f_3 (quitando la medición):

```
qc = deutsch("f3").remove_final_measurements(inplace=False)
psi = Statevector.from_label("00").evolve(qc)
print(np.round(psi.data.real * sqrt(2), 3))      # [-0. -1.  0.  1.]
```

El vector $\frac{1}{\sqrt{2}}(0, -1, 0, 1)$ en la base $|00\rangle, |01\rangle, |10\rangle, |11\rangle$ es $\frac{1}{\sqrt{2}}(-|01\rangle + |11\rangle) = -|-\rangle|1\rangle$, el resultado del Ejercicio 6.1.6.

1.4. Resumen: un ejemplo guiado

Seguimos una sola función, la del ejemplo del libro: $f = f_3$, con $f(0) = 1$ y $f(1) = 0$ (NOT). En cada caja, la parte superior es el cálculo y la inferior, el concepto.

Paso 1. El problema

$f(0) = 1$, $f(1) = 0$, pero no lo sabemos: solo podemos consultar f . ¿Es constante o equilibrada?

Problema de Deutsch: decidir si $f(0) \oplus f(1)$ vale 0 (constante) o 1 (equilibrada). **Modelo de consultas:** el coste es el número de evaluaciones de f .

Paso 2. Clásicamente, dos consultas

Consultamos $f(0) = 1$. Siguen siendo posibles f_4 (constante) y f_3 (equilibrada). Hay que consultar también $f(1)$.

Ningún algoritmo clásico determinista lo resuelve con una consulta.

Paso 3. El oráculo

$U_f |y\rangle |x\rangle = |y \oplus f(x)\rangle |x\rangle$: $U_f |00\rangle = |10\rangle$, $U_f |01\rangle = |01\rangle$, $U_f |10\rangle = |00\rangle$, $U_f |11\rangle = |11\rangle$.

Oráculo: evalúa f de forma reversible guardando la entrada y haciendo XOR en la salida. Es una permutación, $U_f^2 = I$, y es unitaria aunque f no sea invertible.

Paso 4. Primer intento

$U_f |0\rangle |+\rangle = \frac{1}{\sqrt{2}}(|10\rangle + |01\rangle)$: al medir sale $(y, x) = (1, 0)$ o $(0, 1)$, cada uno con probabilidad $\frac{1}{2}$.

Superposición sin interferencia equivale a consultar una entrada al azar: no basta.

Paso 5. El retroceso de fase

$U_f |-\rangle |0\rangle = -|-\rangle |0\rangle$ (pues $f(0) = 1$), $U_f |-\rangle |1\rangle = +|-\rangle |1\rangle$ (pues $f(1) = 0$).

Retroceso de fase: con la salida en $|-\rangle$, el oráculo multiplica $|x\rangle$ por $(-1)^{f(x)}$ y deja la salida igual.

Paso 6. Superposición

$|\varphi_1\rangle = (H \otimes H) |1\rangle |0\rangle = |-\rangle |+\rangle = \frac{1}{2}(|00\rangle + |01\rangle - |10\rangle - |11\rangle)$.

Las Hadamard ponen la entrada en superposición de las dos entradas y la salida en $|-\rangle$. **Orden de Qiskit:** la salida y (abajo) es el factor de la izquierda.

Paso 7. Una consulta

$$|\varphi_2\rangle = |-\rangle \otimes \frac{1}{\sqrt{2}}(-|0\rangle + |1\rangle) = -|-\rangle|-\rangle.$$

Cada componente recibe su signo: la información sobre f está ahora en una **fase relativa**. El estado es un **producto**: no hay entrelazamiento.

Paso 8. Interferencia

$$|\varphi_3\rangle = (I \otimes H)|\varphi_2\rangle = -|-\rangle|1\rangle, \text{ porque } H|-\rangle = |1\rangle: \text{ las amplitudes de } |0\rangle \text{ se cancelan } (\frac{1}{2} - \frac{1}{2} = 0).$$

Interferencia: la Hadamard final convierte la fase relativa en un resultado de la base estándar. Es un **cambio de base**.

Paso 9. Medir

Se obtiene 1 con probabilidad $|-1|^2 = 1$: f es equilibrada. Qiskit: {'1': 1000}.

El resultado es $f(0) \oplus f(1)$ con certeza y **una sola consulta**. El signo -1 (que distinguiría f_3 de f_2) es una fase global: no se observa.

Paso	Qué hicimos	Concepto
1	¿Constante o equilibrada?	Problema de Deutsch, modelo de consultas
2	Consultar $f(0)$	Cota clásica: dos consultas
3	$U_f y\rangle x\rangle = y \oplus f(x)\rangle x\rangle$	Oráculo reversible y unitario
4	Entrada en $ +\rangle$	Superposición sin interferencia no basta
5	Salida en $ -\rangle$	Retroceso de fase
6	$H \otimes H$	Superposición; orden de Qiskit
7	Aplicar U_f	Información en la fase relativa
8	H arriba	Interferencia, cambio de base
9	Medir	$f(0) \oplus f(1)$ con certeza, una consulta

Capítulo 2

El algoritmo de Deutsch–Jozsa

El algoritmo de Deutsch–Jozsa [3, 4] generaliza el de Deutsch a funciones de n bits. Con una sola consulta distingue una función constante de una equilibrada, mientras que un algoritmo clásico determinista necesita $2^{n-1}+1$ consultas: es el primer ejemplo de **ventaja exponencial**. La herramienta nueva es la transformada de Hadamard de n qubits, $H^{\otimes n}$, que aparecerá en todos los capítulos siguientes. Corresponde a la sección 6.2 de Yanofsky.

2.1. El problema y la solución clásica

Consideramos funciones $f : \{0,1\}^n \rightarrow \{0,1\}$, que reciben una cadena de n bits y devuelven un bit. Escribimos las cadenas en negrita, $\mathbf{x} \in \{0,1\}^n$, y a menudo las identificamos con el número natural que representan en binario, de 0 a $2^n - 1$.

Definición: funciones constantes y equilibradas

Una función $f : \{0,1\}^n \rightarrow \{0,1\}$ es **constante** si toma el mismo valor en todas las entradas, y **equilibrada** si toma el valor 0 en exactamente la mitad de las entradas (2^{n-1}) y el valor 1 en la otra mitad.

Ejercicio 6.2.1 (Yanofsky): ¿cuántas funciones hay?

Todas. Para cada una de las 2^n entradas hay que elegir una salida entre dos: hay 2^{2^n} funciones.

Equilibradas. Una función equilibrada queda determinada por el conjunto de las 2^{n-1} entradas que van a 1, que es un subconjunto de tamaño 2^{n-1} de un conjunto de 2^n elementos: hay $\binom{2^n}{2^{n-1}}$.

Constantes. Exactamente 2: la constante 0 y la constante 1.

n	funciones	equilibradas	constantes	ninguna de las dos
1	4	2	2	0
2	16	6	2	8
3	256	70	2	184
4	65 536	12 870	2	52 664

Para $n \geq 2$ la mayoría de las funciones no son ni constantes ni equilibradas: el problema solo tiene sentido con la **promesa** de que f es de uno de los dos tipos.

El problema de Deutsch–Jozsa

Se da $f : \{0,1\}^n \rightarrow \{0,1\}$ como caja negra, con la **promesa** de que es constante o equilibrada. Hay que decidir cuál de las dos cosas es. Para $n = 1$ es el problema de Deutsch.

Proposición: cota clásica

Todo algoritmo clásico determinista que resuelva el problema sin error necesita, en el peor caso, $2^{n-1} + 1$ consultas, y con ese número basta.

Demostración

Bastan. Se consulta f en $2^{n-1} + 1$ entradas distintas. Si aparecen dos valores distintos, f no es constante, luego es equilibrada. Si todos coinciden, f toma el mismo valor en más de la mitad de las entradas, luego no es equilibrada, y por la promesa es constante.

Hacen falta. Supongamos que un algoritmo hace como mucho 2^{n-1} consultas. Lo enfrentamos a un “adversario” que responde 0 a todas. Al terminar, el algoritmo ha visto a lo sumo 2^{n-1} ceros. Hay entonces dos funciones compatibles con todo lo que ha visto: la constante 0, y una función equilibrada que vale 0 en las entradas consultadas (y en otras, si hacen falta para completar 2^{n-1} ceros) y 1 en las 2^{n-1} restantes. La segunda existe porque se han consultado como mucho 2^{n-1} entradas. Las dos funciones producen la misma ejecución, así que el algoritmo da la misma respuesta para ambas y falla en una. $\square$

Nota Aclaratoria: la ventaja exponencial es frente a algoritmos exactos

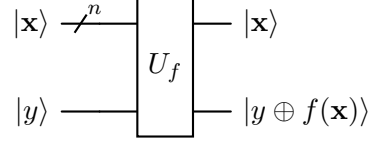
Si se permite un pequeño error, un algoritmo clásico **aleatorio** lo hace muy bien: se consultan k entradas elegidas al azar y se responde “constante” si todas coinciden. Si f es constante, nunca falla. Si f es equilibrada, cada consulta da 0 o 1 con probabilidad $\frac{1}{2}$ cada uno, de forma independiente, así que la probabilidad de que las k coincidan es $2 \cdot (\frac{1}{2})^k = 2^{1-k}$; con $k = 11$ consultas el error es menor que 0,001 para todo n . La ventaja exponencial de Deutsch–Jozsa es, por tanto, frente a algoritmos clásicos que no se equivocan nunca. En el Capítulo 3 (Simon) la ventaja sobrevivirá también frente a los algoritmos aleatorios.

2.2. El algoritmo cuántico

2.2.1. El oráculo de n qubits

El oráculo se define igual que en el Capítulo 1, ahora con un registro de entrada de n qubits [Y 6.44]:

$$U_f(|y\rangle|\mathbf{x}\rangle) = |y \oplus f(\mathbf{x})\rangle|\mathbf{x}\rangle, \quad \mathbf{x} \in \{0,1\}^n, y \in \{0,1\}.$$



La barra con una n indica un registro de n qubits. En Qiskit, $\mathbf{x}$ ocupa $\mathbf{q}_0, \dots, \mathbf{q}_{n-1}$ y y es $\mathbf{q}_n$, de modo que el índice de $|y\rangle|\mathbf{x}\rangle$ en la base es $y \cdot 2^n + \mathbf{x}$. La demostración del Capítulo 1 se aplica literalmente: U_f es una permutación, $U_f^2 = I$ y U_f es unitaria.

Ejemplo 6.2.1 (Yanofsky): el oráculo de una función equilibrada de 2 bits

La función de [Y 6.45] es

$$f(00) = 1, \quad f(01) = 1, \quad f(10) = 0, \quad f(11) = 0,$$

equilibrada (dos ceros y dos unos). En el orden de Qiskit, con la base ordenada como $|y x_1 x_0\rangle$ de $|000\rangle$ a $|111\rangle$, el oráculo intercambia $|0\rangle|\mathbf{x}\rangle \leftrightarrow |1\rangle|\mathbf{x}\rangle$ cuando $f(\mathbf{x}) = 1$ y deja fijos los demás vectores:

$$U_f : |000\rangle \leftrightarrow |100\rangle, \quad |001\rangle \leftrightarrow |101\rangle, \quad |010\rangle, |011\rangle, |110\rangle, |111\rangle \text{ fijos},$$

$$U_f = \begin{matrix} & \begin{matrix} 000 \\ 001 \\ 010 \\ 011 \\ 100 \\ 101 \\ 110 \\ 111 \end{matrix} \end{matrix} \begin{pmatrix} \cdot & \cdot & \cdot & \cdot & 1 & \cdot & \cdot & \cdot \\ \cdot & \cdot & \cdot & \cdot & \cdot & 1 & \cdot & \cdot \\ \cdot & \cdot & 1 & \cdot & \cdot & \cdot & \cdot & \cdot \\ \cdot & \cdot & \cdot & 1 & \cdot & \cdot & \cdot & \cdot \\ 1 & \cdot & \cdot & \cdot & \cdot & \cdot & \cdot & \cdot \\ \cdot & 1 & \cdot & \cdot & \cdot & \cdot & \cdot & \cdot \\ \cdot & \cdot & \cdot & \cdot & \cdot & \cdot & 1 & \cdot \\ \cdot & \cdot & \cdot & \cdot & \cdot & \cdot & \cdot & 1 \end{pmatrix}$$

(los puntos son ceros). La matriz [Y 6.46] del libro intercambia $|00,0\rangle \leftrightarrow |00,1\rangle$ y $|01,0\rangle \leftrightarrow |01,1\rangle$: son los mismos pares de estados con el qubit y escrito a la derecha.

Ejercicios 6.2.2 y 6.2.3 (Yanofsky)

6.2.2. La función de [Y 6.47] es $f(00) = f(01) = 0, f(10) = f(11) = 1$ (equilibrada; $f(\mathbf{x})$ es el primer bit por la izquierda de $\mathbf{x}$). Su oráculo intercambia

$$|010\rangle \leftrightarrow |110\rangle, \quad |011\rangle \leftrightarrow |111\rangle,$$

y deja fijos $|000\rangle, |001\rangle, |100\rangle, |101\rangle$. Es un CNOT con control en $\mathbf{q}_1$ y objetivo en $y = \mathbf{q}_2$.

6.2.3. Para la constante 1, el oráculo cambia siempre y : $|0\rangle|\mathbf{x}\rangle \leftrightarrow |1\rangle|\mathbf{x}\rangle$ para los cuatro $\mathbf{x}$. Es $X \otimes I \otimes I$, la matriz con dos bloques identidad 4×4 fuera de la diagonal, $\begin{pmatrix} 0 & I_4 \\ I_4 & 0 \end{pmatrix}$.

2.2.2. La transformada de Hadamard de n qubits

Para poner n qubits en superposición se aplica una Hadamard a cada uno, es decir, la matriz $H^{\otimes n} = H \otimes \cdots \otimes H$. Yanofsky busca el patrón calculando H , $H^{\otimes 2}$ y $H^{\otimes 3}$ [Y 6.48–6.60]. La clave es escribir las entradas de H como potencias de -1 :

$$H = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix}, \quad H_{ij} = \frac{1}{\sqrt{2}} (-1)^{ij}, \quad i, j \in \{0, 1\},$$

pues el producto ij de dos bits (que es su AND) solo vale 1 en la esquina inferior derecha.

Definición: producto binario de cadenas

Para $\mathbf{x}, \mathbf{z} \in \{0, 1\}^n$,

$$\mathbf{x} \cdot \mathbf{z} = x_{n-1}z_{n-1} \oplus \cdots \oplus x_1z_1 \oplus x_0z_0 \in \{0, 1\}, \quad (\text{Yanofsky: } \langle \mathbf{x}, \mathbf{z} \rangle, [\text{Y 6.54}]).$$

Es la paridad del número de posiciones en que $\mathbf{x}$ y $\mathbf{z}$ valen 1 a la vez. Escribimos también $\mathbf{x} \oplus \mathbf{z}$ para el XOR bit a bit [Y 6.55] y $\mathbf{0}$ para la cadena de n ceros.

Proposición: propiedades del producto binario

Para $\mathbf{x}, \mathbf{x}', \mathbf{z}, \mathbf{z}' \in \{0, 1\}^n$:

1. $(\mathbf{x} \oplus \mathbf{x}') \cdot \mathbf{z} = (\mathbf{x} \cdot \mathbf{z}) \oplus (\mathbf{x}' \cdot \mathbf{z})$ y $\mathbf{x} \cdot (\mathbf{z} \oplus \mathbf{z}') = (\mathbf{x} \cdot \mathbf{z}) \oplus (\mathbf{x} \cdot \mathbf{z}')$ [Y 6.56, 6.57];
2. $\mathbf{0} \cdot \mathbf{z} = \mathbf{x} \cdot \mathbf{0} = 0$ [Y 6.58, 6.59];
3. $\mathbf{x} \cdot \mathbf{z} = \mathbf{z} \cdot \mathbf{x}$;
4. $(-1)^a (-1)^b = (-1)^{a \oplus b}$ para bits a, b .

Demostración

Las propiedades 2 y 3 son inmediatas. Para la 4: $a + b$ y $a \oplus b$ solo difieren cuando $a = b = 1$ (2 frente a 0), y ambos números son pares, así que $(-1)^{a+b} = (-1)^{a \oplus b}$. Para la 1, en cada posición $(x_k \oplus x'_k)z_k = x_k z_k \oplus x'_k z_k$ (es la distributividad en $\mathbb{F}_2$: basta comprobar los ocho casos o notar que, si $z_k = 0$, los dos lados valen 0, y si $z_k = 1$, los dos valen $x_k \oplus x'_k$); sumando con $\oplus$ las n posiciones y reordenando (la operación $\oplus$ es asociativa y conmutativa) se obtiene la igualdad. La segunda igualdad se sigue de la primera y de la simetría.

Comentario. Como observa Yanofsky en una nota a pie de página, $\{0, 1\}^n$ con $\oplus$ es un espacio vectorial sobre el cuerpo de dos elementos $\mathbb{F}_2 = \{0, 1\}$, y $\mathbf{x} \cdot \mathbf{z}$ es una forma bilineal simétrica sobre él. Esta estructura será esencial en el Capítulo 3. $\square$

Teorema: fórmula de $H^{\otimes n}$

Para todo $n \geq 1$ y todas las cadenas $\mathbf{z}$ (fila) y $\mathbf{x}$ (columna),

$$(H^{\otimes n})_{\mathbf{zx}} = \frac{1}{\sqrt{2^n}} (-1)^{\mathbf{x} \cdot \mathbf{z}}. \quad [\text{Y 6.61}]$$

Equivalentemente, para cada cadena $\mathbf{x}$,

$$H^{\otimes n} |\mathbf{x}\rangle = \frac{1}{\sqrt{2^n}} \sum_{\mathbf{z} \in \{0,1\}^n} (-1)^{\mathbf{x} \cdot \mathbf{z}} |\mathbf{z}\rangle \quad [\text{Y 6.64}]$$

y, en particular,

$$H^{\otimes n} |\mathbf{0}\rangle = \frac{1}{\sqrt{2^n}} \sum_{\mathbf{z} \in \{0,1\}^n} |\mathbf{z}\rangle. \quad [\text{Y 6.63}]$$

Demostración

Por inducción sobre n (esto resuelve también el Ejercicio 6.2.4, que pide probar que el factor es $2^{-n/2}$). Para $n = 1$ es la fórmula $H_{ij} = \frac{1}{\sqrt{2}} (-1)^{ij}$.

Supongamos la fórmula cierta para n . Escribimos una cadena de $n + 1$ bits como $\mathbf{x} = x_n \mathbf{x}'$, con x_n el bit de la izquierda y $\mathbf{x}'$ los n restantes, y $H^{\otimes(n+1)} = H \otimes H^{\otimes n}$. Por la definición del producto de Kronecker, la entrada en la fila $z_n \mathbf{z}'$ y la columna $x_n \mathbf{x}'$ es el producto de la entrada (z_n, x_n) del primer factor por la entrada $(\mathbf{z}', \mathbf{x}')$ del segundo:

$$\begin{aligned} (H^{\otimes(n+1)})_{\mathbf{zx}} &= H_{z_n x_n} (H^{\otimes n})_{\mathbf{z}' \mathbf{x}'} = \frac{1}{\sqrt{2}} (-1)^{x_n z_n} \cdot \frac{1}{\sqrt{2^n}} (-1)^{\mathbf{x}' \cdot \mathbf{z}'} \\ &= \frac{1}{\sqrt{2^{n+1}}} (-1)^{x_n z_n \oplus \mathbf{x}' \cdot \mathbf{z}'} = \frac{1}{\sqrt{2^{n+1}}} (-1)^{\mathbf{x} \cdot \mathbf{z}}, \end{aligned}$$

usando la propiedad 4 y que $\mathbf{x} \cdot \mathbf{z} = x_n z_n \oplus \mathbf{x}' \cdot \mathbf{z}'$ por definición. Esto prueba la fórmula para $n + 1$.

La segunda forma es la primera leída por columnas: $H^{\otimes n} |\mathbf{x}\rangle$ es la columna $\mathbf{x}$, cuya entrada $\mathbf{z}$ es $2^{-n/2} (-1)^{\mathbf{x} \cdot \mathbf{z}}$. Con $\mathbf{x} = \mathbf{0}$, todos los signos son $+1$ por la propiedad 2. $\square$

Nota Aclaratoria: la fórmula no depende del orden de los qubits

$H^{\otimes n}$ es la misma matriz con la convención de Yanofsky y con la de Qiskit, porque todos sus factores son iguales; y $\mathbf{x} \cdot \mathbf{z}$ no cambia si se reordenan a la vez las posiciones de $\mathbf{x}$ y de $\mathbf{z}$. Por eso la fórmula [Y 6.61] se puede usar tal cual. Además, $H^{\otimes n}$ es real y simétrica (pues $\mathbf{x} \cdot \mathbf{z} = \mathbf{z} \cdot \mathbf{x}$) y $(H^{\otimes n})^2 = (H^2)^{\otimes n} = I$.

Ejemplo: $H^{\otimes 2}$ y $H^{\otimes 3}$ con números

Con filas y columnas ordenadas de 00 a 11, [Y 6.51]:

$$H^{\otimes 2} = \frac{1}{2} \begin{pmatrix} 1 & 1 & 1 & 1 \\ 1 & -1 & 1 & -1 \\ 1 & 1 & -1 & -1 \\ 1 & -1 & -1 & 1 \end{pmatrix}.$$

Comprobación de una entrada con la fórmula: fila 11, columna 01: $01 \cdot 11 = 0 \oplus 1 = 1$, así que la entrada es $\frac{1}{2}(-1)^1 = -\frac{1}{2}$. Es la cuarta fila y segunda columna, en efecto $-\frac{1}{2}$. Para $n = 3$, $\sqrt{8} H^{\otimes 3}$ es [Y 6.60]

$$\begin{pmatrix} 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 \\ 1 & -1 & 1 & -1 & 1 & -1 & 1 & -1 \\ 1 & 1 & -1 & -1 & 1 & 1 & -1 & -1 \\ 1 & -1 & -1 & 1 & 1 & -1 & -1 & 1 \\ 1 & 1 & 1 & 1 & -1 & -1 & -1 & -1 \\ 1 & -1 & 1 & -1 & -1 & 1 & -1 & 1 \\ 1 & 1 & -1 & -1 & -1 & -1 & 1 & 1 \\ 1 & -1 & -1 & 1 & -1 & 1 & 1 & -1 \end{pmatrix}.$$

Cada fila salvo la primera tiene cuatro +1 y cuatro -1: **suma cero**. Este hecho es el lema siguiente.

Lema: suma de signos

Para todo $\mathbf{z} \in \{0, 1\}^n$,

$$\sum_{\mathbf{x} \in \{0, 1\}^n} (-1)^{\mathbf{x} \cdot \mathbf{z}} = \begin{cases} 2^n & \text{si } \mathbf{z} = \mathbf{0}, \\ 0 & \text{si } \mathbf{z} \neq \mathbf{0}. \end{cases}$$

Demostración

Si $\mathbf{z} = \mathbf{0}$, todos los sumandos valen 1. Si $\mathbf{z} \neq \mathbf{0}$, hay una posición k con $z_k = 1$; sea $\mathbf{e}_k$ la cadena con un 1 solo en la posición k . La aplicación $\mathbf{x} \mapsto \mathbf{x} \oplus \mathbf{e}_k$ es una biyección de $\{0, 1\}^n$ en sí mismo que agrupa las cadenas en parejas $\{\mathbf{x}, \mathbf{x} \oplus \mathbf{e}_k\}$. Por la propiedad 1, $(\mathbf{x} \oplus \mathbf{e}_k) \cdot \mathbf{z} = \mathbf{x} \cdot \mathbf{z} \oplus z_k = \mathbf{x} \cdot \mathbf{z} \oplus 1$: los dos miembros de cada pareja tienen signos opuestos y se cancelan. La suma total es 0. $\square$

El lema dice que las columnas de $\sqrt{2^n} H^{\otimes n}$ son ortogonales a la primera, que es el vector de unos. Es otra forma de ver que $H^{\otimes n}$ es unitaria.

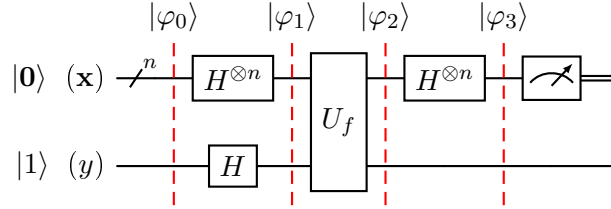
2.2.3. Primer intento

Como en el Capítulo 1, si la entrada es un estado de la base $|\mathbf{x}\rangle$ y la salida está en $|-\rangle$ [Y 6.65], el lema del retroceso de fase (cuya demostración no usa que x sea un solo bit) da

$$U_f(|-\rangle |\mathbf{x}\rangle) = (-1)^{f(\mathbf{x})} |-\rangle |\mathbf{x}\rangle. \quad [\text{Y 6.70}]$$

Es una fase global y no da información. Hay que poner la entrada en superposición.

2.2.4. El algoritmo



En matrices, con el orden de Qiskit, $|\varphi_3\rangle = (I \otimes H^{\otimes n}) U_f (H \otimes H^{\otimes n}) |1\rangle |0\rangle$.

Diferencia con Yanofsky: la fórmula del algoritmo

El libro escribe [Y 6.72] $(H^{\otimes n} \otimes I) U_f (H^{\otimes n} \otimes H) |0, 1\rangle$: los factores de cada producto tensorial aparecen en el orden contrario al nuestro, por la misma razón que en el Capítulo 1.

Teorema: corrección del algoritmo de Deutsch–Jozsa

Para toda $f : \{0, 1\}^n \rightarrow \{0, 1\}$,

$$|\varphi_3\rangle = |-\rangle \otimes \sum_{\mathbf{z} \in \{0, 1\}^n} a_{\mathbf{z}} |\mathbf{z}\rangle, \quad a_{\mathbf{z}} = \frac{1}{2^n} \sum_{\mathbf{x} \in \{0, 1\}^n} (-1)^{f(\mathbf{x}) \oplus \mathbf{x} \cdot \mathbf{z}}.$$

En particular, la probabilidad de obtener $\mathbf{0}$ al medir el registro de arriba es

$$\Pr(\mathbf{0}) = \left(\frac{1}{2^n} \sum_{\mathbf{x}} (-1)^{f(\mathbf{x})} \right)^2 = \left(\frac{N_0 - N_1}{2^n} \right)^2,$$

donde N_0 y N_1 son el número de entradas con $f(\mathbf{x}) = 0$ y $f(\mathbf{x}) = 1$. Por tanto:

- si f es **constante**, se obtiene $\mathbf{0}$ con probabilidad 1;
- si f es **equilibrada**, se obtiene $\mathbf{0}$ con probabilidad 0.

Basta **una consulta** y el resultado es seguro: se responde “constante” si y solo si se mide $\mathbf{0}$.

Demostración

Paso 0. $|\varphi_0\rangle = |1\rangle |\mathbf{0}\rangle$.

Paso 1. Por el teorema de $H^{\otimes n}$, $|\varphi_1\rangle = H|1\rangle \otimes H^{\otimes n}|\mathbf{0}\rangle = |-\rangle \otimes \frac{1}{\sqrt{2^n}} \sum_{\mathbf{x}} |\mathbf{x}\rangle$ [Y 6.74].

Paso 2. Por linealidad y el retroceso de fase, $|\varphi_2\rangle = |-\rangle \otimes \frac{1}{\sqrt{2^n}} \sum_{\mathbf{x}} (-1)^{f(\mathbf{x})} |\mathbf{x}\rangle$ [Y 6.75].

Paso 3. Aplicando $H^{\otimes n}$ a cada $|\mathbf{x}\rangle$ con la fórmula del teorema [Y 6.76]:

$$|\varphi_3\rangle = |-\rangle \otimes \frac{1}{\sqrt{2^n}} \sum_{\mathbf{x}} (-1)^{f(\mathbf{x})} \frac{1}{\sqrt{2^n}} \sum_{\mathbf{z}} (-1)^{\mathbf{x} \cdot \mathbf{z}} |\mathbf{z}\rangle = |-\rangle \otimes \sum_{\mathbf{z}} \left(\frac{1}{2^n} \sum_{\mathbf{x}} (-1)^{f(\mathbf{x}) \oplus \mathbf{x} \cdot \mathbf{z}} \right) |\mathbf{z}\rangle,$$

intercambiando las sumas (finitas) y usando la propiedad 4. Esto es [Y 6.77].

Medición. El estado es un producto con el qubit de abajo en $|-\rangle$, así que la probabilidad de obtener $\mathbf{z}$ arriba es $|a_{\mathbf{z}}|^2$. Para $\mathbf{z} = \mathbf{0}$, $\mathbf{x} \cdot \mathbf{0} = 0$ y $a_{\mathbf{0}} = \frac{1}{2^n} \sum_{\mathbf{x}} (-1)^{f(\mathbf{x})} = \frac{N_0 - N_1}{2^n}$, pues cada entrada con $f(\mathbf{x}) = 0$ aporta +1 y cada una con $f(\mathbf{x}) = 1$ aporta -1. Si f es constante, $N_0 - N_1 = \pm 2^n$ y $a_{\mathbf{0}} = \pm 1$ [Y 6.79, 6.80]; como $\sum_{\mathbf{z}} |a_{\mathbf{z}}|^2 = 1$, las demás amplitudes son 0 y se obtiene $\mathbf{0}$ con certeza. Si f es equilibrada, $N_0 = N_1$ y $a_{\mathbf{0}} = 0$ [Y 6.81]. $\square$

Ejercicio 6.2.5 (Yanofsky): ¿y si nos engañan?

Si f no es ni constante ni equilibrada, el teorema sigue valiendo: se obtiene $\mathbf{0}$ con probabilidad $((N_0 - N_1)/2^n)^2$, que está **estrictamente** entre 0 y 1 (no es 0 porque $N_0 \neq N_1$, ni 1 porque $|N_0 - N_1| < 2^n$). El algoritmo responde a veces “constante” y a veces “equilibrada”, sin que ninguna respuesta tenga sentido.

Ejemplo: $f = \text{AND}$ de dos bits, $f(11) = 1$ y $f = 0$ en las otras tres entradas. Entonces $N_0 = 3$, $N_1 = 1$ y $\Pr(00) = (2/4)^2 = \frac{1}{4}$. Las cuatro amplitudes son

$$a_{\mathbf{z}} = \frac{1}{4} (1 + (-1)^{01 \cdot \mathbf{z}} + (-1)^{10 \cdot \mathbf{z}} - (-1)^{11 \cdot \mathbf{z}}) : \quad a_{00} = a_{01} = a_{10} = \frac{1}{2}, \quad a_{11} = -\frac{1}{2},$$

así que los cuatro resultados salen con probabilidad $\frac{1}{4}$ (Experimento 2).

Ejemplo completo: la función equilibrada

Sea $f(00) = f(01) = 1$, $f(10) = f(11) = 0$, con $n = 2$.

$$\begin{aligned} |\varphi_0\rangle &= |1\rangle |\mathbf{00}\rangle, \\ |\varphi_1\rangle &= |-\rangle \otimes \frac{1}{2} (|\mathbf{00}\rangle + |\mathbf{01}\rangle + |\mathbf{10}\rangle + |\mathbf{11}\rangle), \\ |\varphi_2\rangle &= |-\rangle \otimes \frac{1}{2} (-|\mathbf{00}\rangle - |\mathbf{01}\rangle + |\mathbf{10}\rangle + |\mathbf{11}\rangle). \end{aligned}$$

Para $|\varphi_3\rangle$ calculamos las cuatro amplitudes $a_{\mathbf{z}} = \frac{1}{4} \sum_{\mathbf{x}} (-1)^{f(\mathbf{x})} (-1)^{\mathbf{x} \cdot \mathbf{z}}$, es decir, el producto de la fila $\mathbf{z}$ de $2H^{\otimes 2}$ por el vector de signos $(-1, -1, +1, +1)$:

$$a_{00} = \frac{1}{4} (-1 - 1 + 1 + 1) = 0, \quad a_{01} = \frac{1}{4} (-1 + 1 + 1 - 1) = 0,$$

$$a_{10} = \frac{1}{4} (-1 - 1 - 1 - 1) = -1, \quad a_{11} = \frac{1}{4} (-1 + 1 - 1 + 1) = 0.$$

Así $|\varphi_3\rangle = -|-\rangle |\mathbf{10}\rangle$: se mide 10 con certeza. Como no es 00, f es equilibrada.

Por qué sale justo 10: $(-1)^{f(\mathbf{x})} = -(-1)^{x_1}$, donde x_1 es el primer bit por la izquierda, y $(-1)^{x_1} = (-1)^{\mathbf{x} \cdot \mathbf{10}}$. El vector de signos es, salvo el signo global, una columna de $2H^{\otimes 2}$, y la Hadamard la devuelve al vector de la base $|\mathbf{10}\rangle$.

Conexión con el TFG

La amplitud que decide el algoritmo es un **producto interno**. Con $|s\rangle = H^{\otimes n} |\mathbf{0}\rangle$ y la matriz diagonal de signos $Z_f = \sum_{\mathbf{x}} (-1)^{f(\mathbf{x})} |\mathbf{x}\rangle \langle \mathbf{x}|$,

$$a_{\mathbf{0}} = \langle \mathbf{0} | H^{\otimes n} Z_f H^{\otimes n} | \mathbf{0} \rangle = \langle s | Z_f | s \rangle, \quad \text{Pr}(\mathbf{0}) = |\langle s | Z_f | s \rangle|^2.$$

Es una **fidelidad** entre los estados $|s\rangle$ y $Z_f |s\rangle$, y se estima exactamente como el kernel cuántico $K(\mathbf{x}, \mathbf{y}) = |\langle \phi(\mathbf{x}) | \phi(\mathbf{y}) \rangle|^2$: preparando un estado, deshaciendo otro y midiendo la frecuencia de $|0 \cdots 0\rangle$ (Capítulo 3 del cuaderno). En el **ZZFeatureMap** de Havlíček et al. [11], Z_f se sustituye por una matriz diagonal de fases $e^{i\phi(\mathbf{x})}$ que dependen del dato.

2.3. Implementación en Qiskit

2.3.1. Un oráculo para cualquier función

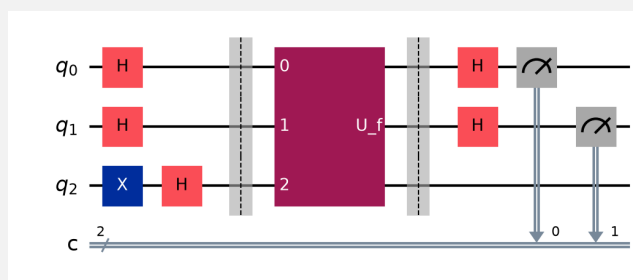
Para funciones de varios bits construimos U_f directamente a partir de su definición, como matriz de permutación. La función recibe y devuelve enteros, que son las cadenas de bits leídas en binario al modo de Qiskit:

```
def oraculo(f, n, m, etiqueta="U_f"):
    """U_f |y>|x> = |y xor f(x)>|x>: x en los n primeros qubits, y en
        los m siguientes."""
    N = 2 ** (n + m)
    U = np.zeros((N, N))
    for x in range(2 ** n):
        for y in range(2 ** m):
            U[(y ^ f(x)) * 2 ** n + x, y * 2 ** n + x] = 1
    return UnitaryGate(U, label=etiqueta)
```

La entrada (fila, columna) = $((y \oplus f(x)) 2^n + x, y 2^n + x)$ es exactamente “la columna de $|y\rangle |x\rangle$ tiene su 1 en la fila de $|y \oplus f(x)\rangle |x\rangle$ ”. Al añadir la puerta con `qc.append(oraculo(...), range(n + m))`, el primer qubit de la lista es el bit menos significativo del índice, de acuerdo con el orden de Qiskit. (Construir así el oráculo no es eficiente para n grande, pues la matriz tiene 4^{n+m} entradas, pero es perfecto para comprobar el algoritmo; en el Capítulo 4 se construye uno con puertas.)

2.3.2. Experimento 2: Deutsch–Jozsa con funciones de 2 bits

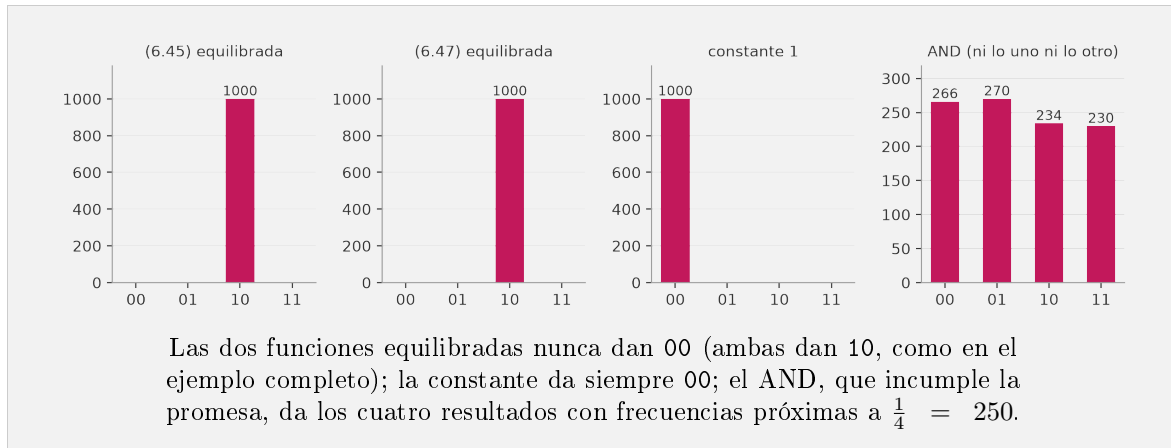
```
def deutsch_jozsa(f, n):
    qc = QuantumCircuit(n + 1, n)
    qc.x(n)
    qc.h(range(n + 1))
    qc.barrier()
    qc.append(oraculo(f, n, 1), range(n + 1))
    qc.barrier()
    qc.h(range(n))
    qc.measure(range(n), range(n))
    return qc
```



El circuito de Deutsch–Jozsa con $n = 2$: q_0, q_1 forman el registro x y q_2 es y .

Lo ejecutamos con las funciones [Y 6.45] y [Y 6.47], con la constante 1 y con el AND del Ejercicio 6.2.5, con 1000 disparos:

```
(6.45) equilibrada {'10': 1000}
(6.47) equilibrada {'10': 1000}
constante 1 {'00': 1000}
AND (ni lo uno ni lo otro) {'11': 230, '10': 234, '00': 266, '01': 270}
```



Las probabilidades exactas, calculadas con el método `probabilities_dict` de `Statevector`, son 1 para 10 en las dos equilibradas, 1 para 00 en la constante y 0,25 para cada resultado con el AND. También se comprueba que el estado final de [Y 6.45] es exactamente $|-\rangle \otimes (-|10\rangle)$:

```
esperado = Statevector(np.kron(np.array([1, -1]) / sqrt(2), -np.eye(4)
[2]))
print(Statevector.from_label("000").evolve(qc).equiv(esperado)) #
True
```

2.3.3. Experimento 3: una función equilibrada de 3 bits al azar

Elegimos al azar una función equilibrada de 3 bits (una permutación aleatoria de cuatro ceros y cuatro unos). Resultó ser

$$f = (000 \mapsto 0, 001 \mapsto 1, 010 \mapsto 1, 011 \mapsto 0, 100 \mapsto 1, 101 \mapsto 1, 110 \mapsto 0, 111 \mapsto 0).$$

Con 1000 disparos se obtuvo $\{'011': 237, '010': 239, '110': 253, '111': 271\}$: nunca 000, como predice el teorema. La constante 0 dio $\{'000': 1000\}$.

2.4. Resumen: un ejemplo guiado

Seguimos la función equilibrada de [Y 6.45]: $f(00) = f(01) = 1$, $f(10) = f(11) = 0$.

Paso 1. El problema

$f : \{0,1\}^2 \rightarrow \{0,1\}$, con la promesa de que es constante o equilibrada. De las 16 funciones de 2 bits, 2 son constantes y 6 equilibradas.

Problema de Deutsch–Jozsa (con promesa). Funciones equilibradas: $\binom{2^n}{2^{n-1}}$.

Paso 2. Clásicamente

Consultamos $f(10) = 0$ y $f(11) = 0$: todavía puede ser la constante 0. Hace falta una tercera consulta: $2^{n-1} + 1 = 3$.

Cota clásica determinista: $2^{n-1} + 1$ consultas (argumento del adversario). Con azar y un error pequeño bastan unas pocas.

Paso 3. El oráculo

U_f intercambia $|000\rangle \leftrightarrow |100\rangle$ y $|001\rangle \leftrightarrow |101\rangle$ (las entradas con $f = 1$) y deja fijos los otros cuatro.

$U_f |y\rangle |\mathbf{x}\rangle = |y \oplus f(\mathbf{x})\rangle |\mathbf{x}\rangle$: permutación, involución, unitaria. En Qiskit, $\mathbf{x}$ arriba (q_0, q_1) e y abajo (q_2) .

Paso 4. Superposición uniforme

$|\varphi_1\rangle = |-\rangle \otimes \frac{1}{2}(|00\rangle + |01\rangle + |10\rangle + |11\rangle)$.

$H^{\otimes n} |\mathbf{0}\rangle = 2^{-n/2} \sum_{\mathbf{x}} |\mathbf{x}\rangle$: la primera columna de $H^{\otimes n}$ es toda de unos.

Paso 5. Una consulta

$|\varphi_2\rangle = |-\rangle \otimes \frac{1}{2}(-|00\rangle - |01\rangle + |10\rangle + |11\rangle)$.

Retroceso de fase en todas las componentes a la vez: los signos son $(-1)^{f(\mathbf{x})}$.

Paso 6. La fórmula de $H^{\otimes n}$

Fila 10 de $2H^{\otimes 2}$: $(1, 1, -1, -1)$, pues $00 \cdot 10 = 01 \cdot 10 = 0$ y $10 \cdot 10 = 11 \cdot 10 = 1$.

$(H^{\otimes n})_{\mathbf{z}\mathbf{x}} = 2^{-n/2}(-1)^{\mathbf{x} \cdot \mathbf{z}}$, con el **producto binario** $\mathbf{x} \cdot \mathbf{z}$ (paridad de los unos comunes). Se demuestra por inducción.

Paso 7. Interferencia

$$a_{00} = \frac{1}{4}(-1 - 1 + 1 + 1) = 0, \quad a_{10} = \frac{1}{4}(-1 - 1 - 1 - 1) = -1, \quad a_{01} = a_{11} = 0.$$

Lema de la suma de signos: $\sum_{\mathbf{x}} (-1)^{\mathbf{x} \cdot \mathbf{z}} = 0$ si $\mathbf{z} \neq \mathbf{0}$. La amplitud de $\mathbf{0}$ es la media de los signos: $\frac{N_0 - N_1}{2^n}$.

Paso 8. Medir

Sale 10 con certeza (Qiskit: $\{\text{'10': 1000}\}$). No es 00: f es equilibrada.

Constante $\iff$ se mide $\mathbf{0}$ con probabilidad 1; **equilibrada** $\iff$ probabilidad 0. Una consulta frente a $2^{n-1} + 1$.

Paso 9. Si incumplen la promesa

$$\text{AND: } \Pr(00) = \left(\frac{3-1}{4}\right)^2 = \frac{1}{4}.$$

Sin la promesa el algoritmo no decide nada: $\Pr(\mathbf{0}) = \left(\frac{N_0 - N_1}{2^n}\right)^2$ está estrictamente entre 0 y 1.

Paso	Qué hicimos	Concepto
1	¿Constante o equilibrada?	Problema con promesa; contar funciones
2	Consultas clásicas	Cota $2^{n-1} + 1$ (adversario)
3	U_f de 3 qubits	Oráculo de n qubits; orden de Qiskit
4	$H^{\otimes 2} 00\rangle$	Superposición uniforme
5	Aplicar U_f	Retroceso de fase en paralelo
6	Fila de $H^{\otimes 2}$	Fórmula de $H^{\otimes n}$, producto binario
7	Amplitudes de $ \varphi_3\rangle$	Interferencia, suma de signos
8	Medir 10	Una consulta, sin error
9	AND	El papel de la promesa

Capítulo 3

El algoritmo de periodicidad de Simon

El algoritmo de Simon [5] busca un **patrón oculto** en una función: una cadena $\mathbf{c}$ tal que f toma el mismo valor en $\mathbf{x}$ y en $\mathbf{x} \oplus \mathbf{c}$. Es el primer algoritmo de este cuaderno que **combina** una parte cuántica, que se repite varias veces y da resultados aleatorios, con una parte **clásica**, que resuelve un sistema de ecuaciones lineales sobre el cuerpo de dos elementos. Su ventaja es exponencial incluso frente a algoritmos clásicos aleatorios, y su estructura (encontrar un período con la transformada de Hadamard o de Fourier) es la que después usa Shor. Corresponde a la sección 6.3 de Yanofsky.

3.1. El problema y la solución clásica

El problema de Simon

Se da $f : \{0, 1\}^n \rightarrow \{0, 1\}^n$ como caja negra, con la promesa de que existe una cadena $\mathbf{c} \in \{0, 1\}^n$ tal que, para todas $\mathbf{x}, \mathbf{y} \in \{0, 1\}^n$,

$$f(\mathbf{x}) = f(\mathbf{y}) \iff \mathbf{y} = \mathbf{x} \text{ o } \mathbf{y} = \mathbf{x} \oplus \mathbf{c}.$$

La cadena $\mathbf{c}$ se llama **período** de f . Hay que encontrar $\mathbf{c}$.

Nota Aclaratoria: el enunciado del libro

Yanofsky escribe la condición como “ $f(\mathbf{x}) = f(\mathbf{y})$ si y solo si $\mathbf{x} = \mathbf{y} \oplus \mathbf{c}$ ” [Y 6.82]. Tomada al pie de la letra, con $\mathbf{x} = \mathbf{y}$ y $\mathbf{c} \neq \mathbf{0}$ diría que $f(\mathbf{x}) \neq f(\mathbf{x})$, lo cual es absurdo; la lectura correcta es la de arriba, que incluye el caso trivial $\mathbf{y} = \mathbf{x}$. Para $\mathbf{c} = \mathbf{0}$ las dos lecturas coinciden.

Proposición: forma de las funciones de Simon

Si $\mathbf{c} = \mathbf{0}$, f es inyectiva (y, por tanto, biyectiva). Si $\mathbf{c} \neq \mathbf{0}$, f es **dos a uno**: cada valor de la imagen tiene exactamente dos preimágenes, de la forma $\{\mathbf{x}, \mathbf{x} \oplus \mathbf{c}\}$, y la imagen tiene 2^{n-1} elementos.

Demostración

Si $\mathbf{c} = \mathbf{0}$, la promesa dice que $f(\mathbf{x}) = f(\mathbf{y})$ solo si $\mathbf{y} = \mathbf{x}$: f es inyectiva, y una aplicación inyectiva de un conjunto finito en sí mismo es biyectiva. Si $\mathbf{c} \neq \mathbf{0}$, para cada $\mathbf{x}$ las cadenas $\mathbf{x}$ y $\mathbf{x} \oplus \mathbf{c}$ son distintas, tienen la misma imagen, y ninguna otra cadena tiene esa imagen. Las parejas $\{\mathbf{x}, \mathbf{x} \oplus \mathbf{c}\}$ forman una partición de $\{0, 1\}^n$ (la relación “ $\mathbf{y} = \mathbf{x}$ o $\mathbf{y} = \mathbf{x} \oplus \mathbf{c}$ ” es de equivalencia, porque $(\mathbf{x} \oplus \mathbf{c}) \oplus \mathbf{c} = \mathbf{x}$), con $2^n/2 = 2^{n-1}$ clases, y f envía clases distintas a valores distintos. $\square$

Ejemplo 6.3.1 (Yanofsky) y Ejercicio 6.3.1

Con $\mathbf{c} = 101$ ($n = 3$): las parejas $\{\mathbf{x}, \mathbf{x} \oplus 101\}$ son

$$\{000, 101\}, \quad \{001, 100\}, \quad \{010, 111\}, \quad \{011, 110\},$$

y la promesa exige $f(000) = f(101)$, $f(001) = f(100)$, $f(010) = f(111)$ y $f(011) = f(110)$, con cuatro valores distintos. (El libro lista las ocho igualdades, pero cada una aparece dos veces.)

Con $\mathbf{c} = 011$ (Ejercicio 6.3.1): $000 \oplus 011 = 011$, $001 \oplus 011 = 010$, $100 \oplus 011 = 111$ y $101 \oplus 011 = 110$, así que

$$f(000) = f(011), \quad f(001) = f(010), \quad f(100) = f(111), \quad f(101) = f(110),$$

y los cuatro valores tienen que ser distintos entre sí.

Proposición: solución clásica

Clásicamente, $\mathbf{c}$ se encuentra en cuanto se observa una **colisión**, dos entradas distintas $\mathbf{x}_1 \neq \mathbf{x}_2$ con $f(\mathbf{x}_1) = f(\mathbf{x}_2)$: entonces $\mathbf{c} = \mathbf{x}_1 \oplus \mathbf{x}_2$ [Y 6.85]. Consultando $2^{n-1} + 1$ entradas distintas se garantiza encontrar una colisión si $\mathbf{c} \neq \mathbf{0}$, y si no aparece ninguna se concluye que $\mathbf{c} = \mathbf{0}$.

Demostración

Si $f(\mathbf{x}_1) = f(\mathbf{x}_2)$ con $\mathbf{x}_1 \neq \mathbf{x}_2$, la promesa da $\mathbf{x}_2 = \mathbf{x}_1 \oplus \mathbf{c}$; aplicando $\oplus \mathbf{x}_1$ a los dos lados, $\mathbf{x}_1 \oplus \mathbf{x}_2 = \mathbf{c}$ (porque $\mathbf{x}_1 \oplus \mathbf{x}_1 = \mathbf{0}$). Si $\mathbf{c} \neq \mathbf{0}$, la imagen de f tiene 2^{n-1} elementos; entre $2^{n-1} + 1$ valores de f hay dos iguales por el principio del palomar. Si no hay colisión entre $2^{n-1} + 1$ entradas, $\mathbf{c} \neq \mathbf{0}$ es imposible. $\square$

Nota Aclaratoria: ¿y con un algoritmo aleatorio?

Consultando entradas al azar se encuentra una colisión mucho antes, por la **paradoja del cumpleaños**: con unas $2^{n/2}$ consultas la probabilidad de colisión ya es apreciable. Simon demostró que ningún algoritmo clásico, ni siquiera aleatorio, puede hacerlo esencialmente mejor: necesita del orden de $2^{n/2}$ consultas [5]. Como el algoritmo cuántico usa del orden de n , la ventaja es exponencial también frente a los algoritmos aleatorios, a diferencia de Deutsch–Jozsa.

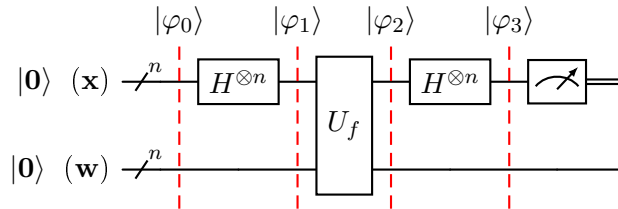
3.2. El algoritmo cuántico

3.2.1. El circuito

El oráculo tiene ahora un registro de salida de n qubits y hace el XOR bit a bit [Y 6.83]:

$$U_f(|\mathbf{w}\rangle|\mathbf{x}\rangle) = |\mathbf{w} \oplus f(\mathbf{x})\rangle|\mathbf{x}\rangle, \quad \mathbf{x}, \mathbf{w} \in \{0, 1\}^n.$$

Como en los capítulos anteriores, es una permutación y su propia inversa. En Qiskit, $\mathbf{x}$ ocupa $\mathbf{q}_0, \dots, \mathbf{q}_{n-1}$ (arriba) y $\mathbf{w}$ ocupa $\mathbf{q}_n, \dots, \mathbf{q}_{2n-1}$ (abajo). La parte cuántica del algoritmo es [Y 6.86]



es decir, $|\varphi_3\rangle = (I \otimes H^{\otimes n}) U_f (I \otimes H^{\otimes n}) |\mathbf{0}\rangle |\mathbf{0}\rangle$ (el libro: $(H^{\otimes n} \otimes I) U_f (H^{\otimes n} \otimes I) |\mathbf{0}, \mathbf{0}\rangle$, [Y 6.87]). Obsérvese que esta vez el registro de salida empieza en $|\mathbf{0}\rangle$, no en $|-\rangle$: aquí no se usa el retroceso de fase, sino el entrelazamiento entre los dos registros.

Teorema: distribución de los resultados

Al medir el registro de arriba de $|\varphi_3\rangle$ se obtiene una cadena $\mathbf{z}$ con probabilidad

$$\Pr(\mathbf{z}) = \begin{cases} \frac{1}{2^{n-1}} & \text{si } \mathbf{z} \cdot \mathbf{c} = 0, \\ 0 & \text{si } \mathbf{z} \cdot \mathbf{c} = 1, \end{cases} \quad \text{cuando } \mathbf{c} \neq \mathbf{0},$$

y $\Pr(\mathbf{z}) = 2^{-n}$ para todo $\mathbf{z}$ cuando $\mathbf{c} = \mathbf{0}$. Es decir: **cada ejecución devuelve una cadena uniformemente al azar entre las que cumplen $\mathbf{z} \cdot \mathbf{c} = 0$.**

Demostración

Los estados. Como en Deutsch–Jozsa,

$$|\varphi_1\rangle = \frac{1}{\sqrt{2^n}} \sum_{\mathbf{x}} |\mathbf{0}\rangle |\mathbf{x}\rangle, \quad |\varphi_2\rangle = \frac{1}{\sqrt{2^n}} \sum_{\mathbf{x}} |f(\mathbf{x})\rangle |\mathbf{x}\rangle \quad [\text{Y 6.90}],$$

$$|\varphi_3\rangle = \frac{1}{2^n} \sum_{\mathbf{x}} \sum_{\mathbf{z}} (-1)^{\mathbf{x} \cdot \mathbf{z}} |f(\mathbf{x})\rangle |\mathbf{z}\rangle \quad [\text{Y 6.91}].$$

Caso $\mathbf{c} \neq \mathbf{0}$. Agrupamos los sumandos por el valor $\mathbf{w} = f(\mathbf{x})$. Por la proposición anterior, cada $\mathbf{w}$ de la imagen tiene exactamente dos preimágenes, $\mathbf{x}_{\mathbf{w}}$ y $\mathbf{x}_{\mathbf{w}} \oplus \mathbf{c}$, así que el coeficiente de $|\mathbf{w}\rangle |\mathbf{z}\rangle$ es

$$\frac{1}{2^n} \left((-1)^{\mathbf{x}_{\mathbf{w}} \cdot \mathbf{z}} + (-1)^{(\mathbf{x}_{\mathbf{w}} \oplus \mathbf{c}) \cdot \mathbf{z}} \right) = \frac{(-1)^{\mathbf{x}_{\mathbf{w}} \cdot \mathbf{z}}}{2^n} \left(1 + (-1)^{\mathbf{c} \cdot \mathbf{z}} \right) \quad [\text{Y 6.93}],$$

usando $(\mathbf{x}_{\mathbf{w}} \oplus \mathbf{c}) \cdot \mathbf{z} = \mathbf{x}_{\mathbf{w}} \cdot \mathbf{z} \oplus \mathbf{c} \cdot \mathbf{z}$ y $(-1)^{a \oplus b} = (-1)^a (-1)^b$. Si $\mathbf{c} \cdot \mathbf{z} = 1$, el paréntesis vale 0: **las dos contribuciones interfieren destructivamente**. Si $\mathbf{c} \cdot \mathbf{z} = 0$, el coeficiente vale $\pm 2/2^n$. La probabilidad de medir $\mathbf{z}$ arriba es la suma de los cuadrados de los coeficientes de $|\mathbf{w}\rangle |\mathbf{z}\rangle$ sobre todos los $\mathbf{w}$; si $\mathbf{c} \cdot \mathbf{z} = 0$ hay 2^{n-1} valores $\mathbf{w}$ en la imagen, y

$$\text{Pr}(\mathbf{z}) = 2^{n-1} \cdot \left(\frac{2}{2^n} \right)^2 = \frac{2^{n-1} \cdot 4}{4^n} = \frac{1}{2^{n-1}}.$$

Comprobación de que suman 1. Por el lema de la suma de signos, el número de cadenas con $\mathbf{z} \cdot \mathbf{c} = 0$ es $\sum_{\mathbf{z}} \frac{1+(-1)^{\mathbf{z} \cdot \mathbf{c}}}{2} = \frac{2^n+0}{2} = 2^{n-1}$, y $2^{n-1} \cdot 2^{-(n-1)} = 1$.

Caso $\mathbf{c} = \mathbf{0}$. Ahora f es inyectiva, los kets $|f(\mathbf{x})\rangle |\mathbf{z}\rangle$ son todos distintos y cada uno tiene coeficiente $\pm 2^{-n}$. Para cada $\mathbf{z}$ hay 2^n de ellos, luego $\text{Pr}(\mathbf{z}) = 2^n \cdot 4^{-n} = 2^{-n}$. $\square$

Nota Aclaratoria: la intuición

Si se midiera primero el registro de abajo (lo que no cambia la distribución del de arriba, porque las operaciones posteriores solo actúan arriba) y saliera $\mathbf{w}$, el registro de arriba quedaría en $\frac{1}{\sqrt{2}}(|\mathbf{x}_{\mathbf{w}}\rangle + |\mathbf{x}_{\mathbf{w}} \oplus \mathbf{c}\rangle)$: una superposición de las dos únicas entradas con ese valor, que “contiene” $\mathbf{c}$ en la diferencia. Pero medir este estado daría $\mathbf{x}_{\mathbf{w}}$ o $\mathbf{x}_{\mathbf{w}} \oplus \mathbf{c}$ al azar, sin revelar $\mathbf{c}$. La $H^{\otimes n}$ final transforma ese “desplazamiento por $\mathbf{c}$ ” en una **restricción sobre el resultado**, $\mathbf{z} \cdot \mathbf{c} = 0$, que es la misma para todos los $\mathbf{w}$. Es el análogo binario de un hecho de la transformada de Fourier: una función periódica tiene la transformada concentrada en las frecuencias “ortogonales” al período.

Ejemplo completo: la función f , con $\mathbf{c} = 101$

La función es

$\mathbf{x}$	000	001	010	011	100	101	110	111
$f(\mathbf{x})$	100	001	101	111	001	100	111	101

Las parejas con el mismo valor son $\{000, 101\} \mapsto 100$, $\{001, 100\} \mapsto 001$, $\{010, 111\} \mapsto 101$ y $\{011, 110\} \mapsto 111$: en todas, $\mathbf{x} \oplus \mathbf{x}' = 101$.

Estados. $|\varphi_1\rangle = \frac{1}{\sqrt{8}} \sum_{\mathbf{x}} |000\rangle |\mathbf{x}\rangle$ y $|\varphi_2\rangle = \frac{1}{\sqrt{8}} (|100\rangle |000\rangle + |001\rangle |001\rangle + |101\rangle |010\rangle + \dots + |101\rangle |111\rangle)$.

Un coeficiente a mano. El coeficiente de $|101\rangle |010\rangle$ en $|\varphi_3\rangle$ (valor $\mathbf{w} = 101$, resultado $\mathbf{z} = 010$) recibe las contribuciones de $\mathbf{x} = 010$ y $\mathbf{x} = 111$:

$$\frac{1}{8}((-1)^{010 \cdot 010} + (-1)^{111 \cdot 010}) = \frac{1}{8}((-1)^1 + (-1)^1) = -\frac{1}{4}.$$

El de $|101\rangle |001\rangle$: $\frac{1}{8}((-1)^{010 \cdot 001} + (-1)^{111 \cdot 001}) = \frac{1}{8}(1 - 1) = 0$, y en efecto $001 \cdot 101 = 1$.

Resultado. Haciendo las 64 cuentas (Yanofsky las escribe todas en las páginas 191–192) sobreviven solo los $\mathbf{z}$ con $\mathbf{z} \cdot 101 = 0$:

$$\begin{aligned} |\varphi_3\rangle = \frac{1}{4} \Big[& (|100\rangle + |001\rangle + |101\rangle + |111\rangle) |000\rangle \\ & + (|100\rangle + |001\rangle - |101\rangle - |111\rangle) |010\rangle \\ & + (|100\rangle - |001\rangle + |101\rangle - |111\rangle) |101\rangle \\ & + (|100\rangle - |001\rangle - |101\rangle + |111\rangle) |111\rangle \Big]. \end{aligned}$$

Cada uno de los cuatro resultados 000, 010, 101, 111 sale con probabilidad $4 \cdot \frac{1}{16} = \frac{1}{4} = 2^{-(3-1)}$, como predice el teorema. (Es la última expresión de la página 192 del libro, con los registros en el orden de Qiskit.)

Encontrar $\mathbf{c}$. Escribimos $\mathbf{c} = c_2 c_1 c_0$ (c_0 es el bit de la derecha). Las ecuaciones $\mathbf{z} \cdot \mathbf{c} = 0$ son

$$000 \cdot \mathbf{c} = 0 \text{ (no dice nada)}, \quad 010 \cdot \mathbf{c} = c_1 = 0,$$

$$101 \cdot \mathbf{c} = c_2 \oplus c_0 = 0, \quad 111 \cdot \mathbf{c} = c_2 \oplus c_1 \oplus c_0 = 0.$$

De la segunda, $c_1 = 0$; de la tercera, $c_2 = c_0$. Las soluciones son 000 y 101, y como f no es inyectiva (por ejemplo, $f(000) = f(101)$), $\mathbf{c} = 101$.

Ejercicio 6.3.2 (Yanofsky): la función

La función es

$\mathbf{x}$	000	001	010	011	100	101	110	111
$f(\mathbf{x})$	000	100	100	000	010	110	110	010

Parejas: $\{000, 011\} \mapsto 000$, $\{001, 010\} \mapsto 100$, $\{100, 111\} \mapsto 010$, $\{101, 110\} \mapsto 110$; en todas la diferencia es 011, así que $\mathbf{c} = 011$. Las cadenas con $\mathbf{z} \cdot 011 = 0$ son 000, 011, 100, 111, y

$$\begin{aligned} |\varphi_3\rangle = \frac{1}{4} & \left[(|000\rangle + |100\rangle + |010\rangle + |110\rangle) |000\rangle \right. \\ & + (|000\rangle + |100\rangle - |010\rangle - |110\rangle) |011\rangle \\ & + (|000\rangle - |100\rangle + |010\rangle - |110\rangle) |100\rangle \\ & \left. + (|000\rangle - |100\rangle - |010\rangle + |110\rangle) |111\rangle \right]. \end{aligned}$$

Por ejemplo, el coeficiente de $|010\rangle |011\rangle$ viene de $\mathbf{x} = 100$ y 111 : $\frac{1}{8}((-1)^0 + (-1)^0) = \frac{1}{4}$; el de $|100\rangle |011\rangle$ viene de 001 y 010 : $\frac{1}{8}((-1)^1 + (-1)^1) = -\frac{1}{4}$. Con los resultados 100 y 011: $c_2 = 0$ y $c_1 \oplus c_0 = 0$, luego $\mathbf{c} \in \{000, 011\}$ y $\mathbf{c} = 011$.

3.2.2. La parte clásica: ecuaciones lineales sobre $\mathbb{F}_2$

Cada ejecución da una ecuación lineal $\mathbf{z} \cdot \mathbf{c} = 0$ para las n incógnitas $c_{n-1}, \dots, c_0 \in \{0, 1\}$. Es un sistema lineal homogéneo sobre el cuerpo $\mathbb{F}_2 = \{0, 1\}$ (con $\oplus$ como suma), y se resuelve por **eliminación gaussiana** exactamente como sobre $\mathbb{R}$: “sumar una ecuación a otra” es hacer el XOR de las cadenas, y no hace falta dividir porque el único coeficiente no nulo es 1.

Proposición: cuándo el sistema determina $\mathbf{c}$

Sea $\mathbf{c} \neq \mathbf{0}$ y sean $\mathbf{z}_1, \dots, \mathbf{z}_k$ cadenas con $\mathbf{z}_i \cdot \mathbf{c} = 0$. Si el subespacio W que generan tiene dimensión $n - 1$, el conjunto de soluciones de $\{\mathbf{z}_i \cdot \mathbf{u} = 0\}_{i=1}^k$ es exactamente $\{\mathbf{0}, \mathbf{c}\}$.

Demostración

Sea Z la matriz $k \times n$ sobre $\mathbb{F}_2$ cuyas filas son las $\mathbf{z}_i$. Las soluciones son el núcleo de Z , y el rango de Z es $\dim W = n - 1$. Por el teorema del rango (válido sobre cualquier cuerpo), el núcleo tiene dimensión $n - (n - 1) = 1$, así que tiene $2^1 = 2$ elementos. Como $\mathbf{0}$ y $\mathbf{c}$ son soluciones y son distintos, son todas. $\square$

Ejemplo 6.3.2 (Yanofsky): eliminación con $n = 7$

Siete ejecuciones dan las ecuaciones $\mathbf{z}_i \cdot \mathbf{c} = 0$ con

$$\begin{aligned} & \text{(i) } 1010110, \text{ (ii) } 0010001, \text{ (iii) } 1100101, \text{ (iv) } 0011011, \\ & \text{(v) } 0101001, \text{ (vi) } 0011010, \text{ (vii) } 0110111. \end{aligned}$$

Los pasos del libro (comprobados uno a uno con código) son:

1. (iii) $\oplus$ = (i): (iii) pasa a 0110011 (se limpia la primera columna).
2. (v) $\oplus$ = (iii) y (vii) $\oplus$ = (iii): (v) = 0011010, (vii) = 0000100.
3. (i), (iii), (iv), (v), (vi) $\oplus$ = (ii): 1000111, 0100010, 0001010, 0001011, 0001011.
4. (v), (vi) $\oplus$ = (iv) y (i) $\oplus$ = (vii): (v) = (vi) = 0000001, (i) = 1000011.
5. (i), (ii), (vi) $\oplus$ = (v): (i) = 1000010, (ii) = 0010000, (vi) = 0000000.

Numerando los bits de $\mathbf{c}$ de izquierda a derecha como hace el libro, $\mathbf{c} = d_1 d_2 \cdots d_7$, queda

$$d_1 \oplus d_6 = 0, \quad d_3 = 0, \quad d_2 \oplus d_6 = 0, \quad d_4 \oplus d_6 = 0, \quad d_7 = 0, \quad d_5 = 0.$$

Hay 6 ecuaciones independientes (la (vi) se anuló: era combinación de las demás), el rango es $6 = n - 1$ y las soluciones son $d_6 = 0$ (todo ceros) o $d_6 = 1$, que da $\mathbf{c} = 1101010$.

Una corrección al libro: el Ejercicio 6.3.3 no tiene solución única

El Ejercicio 6.3.3 pide resolver

$$\begin{aligned} & 11110000, 01101001, 10010110, 00111100, \\ & 11111111, 11000011, 10001110, 01110001, \end{aligned}$$

y da como pista $\mathbf{c} = 10011001$. Esa cadena cumple las ocho ecuaciones, pero **no es la única**: el sistema tiene rango 5, no 7, y por el teorema del rango sus soluciones forman un subespacio de dimensión $8 - 5 = 3$, con 8 elementos. Las siete soluciones no nulas son

$$10011001, 01011010, 11000011, 00111100, 10100101, 01100110, 11111111.$$

El motivo es que tres de las ecuaciones son combinaciones de otras:

$$(v) = (ii) \oplus (iii), \quad (vi) = (iv) \oplus (v), \quad (viii) = (v) \oplus (vii),$$

por ejemplo $01101001 \oplus 10010110 = 11111111$. Quedan $8 - 3 = 5$ ecuaciones independientes. En una ejecución real del algoritmo de Simon se seguiría ejecutando el circuito hasta que el rango llegara a $n - 1 = 7$; con estas ocho ecuaciones todavía no se puede afirmar cuál es $\mathbf{c}$.

3.2.3. ¿Cuántas ejecuciones hacen falta?

Yanofsky afirma que el período se encuentra “con n evaluaciones de la función” [Y p. 195]. No es exactamente así: los resultados son aleatorios y pueden repetirse o ser combinaciones de los anteriores (en el Experimento 4 una ejecución da tres veces seguidas el mismo resultado).

Lo correcto es que el número **medio** de ejecuciones es menor que $n + 1$, y se demuestra a continuación.

Teorema: número de ejecuciones del algoritmo de Simon

Sea $\mathbf{c} \neq \mathbf{0}$ y sea $V = \{\mathbf{z} : \mathbf{z} \cdot \mathbf{c} = 0\}$, un subespacio de dimensión $n - 1$. Si se ejecuta la parte cuántica hasta que los resultados generen V :

1. la probabilidad de que basten exactamente $n - 1$ ejecuciones es $\prod_{j=1}^{n-1} (1 - 2^{-j}) > \frac{1}{4}$;
2. el número medio de ejecuciones es $\sum_{j=1}^{n-1} \frac{1}{1 - 2^{-j}} < n + 1$.

Demostración

Por el teorema anterior, cada ejecución devuelve un elemento de V uniformemente al azar, independientemente de las demás. Supongamos que los resultados obtenidos hasta ahora generan un subespacio $W \subseteq V$ de dimensión $r < n - 1$. La siguiente ejecución aumenta la dimensión si y solo si su resultado no está en W , lo que ocurre con probabilidad

$$p_r = 1 - \frac{|W|}{|V|} = 1 - \frac{2^r}{2^{n-1}} = 1 - 2^{-(n-1-r)}.$$

Apartado 1. Bastan $n - 1$ ejecuciones si y solo si cada una aumenta la dimensión, con $r = 0, 1, \dots, n - 2$; la probabilidad es $\prod_{r=0}^{n-2} p_r = \prod_{j=1}^{n-1} (1 - 2^{-j})$ (cambiando $j = n - 1 - r$). Para acotarla usamos que $(1 - a)(1 - b) \geq 1 - a - b$ si $a, b \geq 0$, que por inducción da $\prod_k (1 - a_k) \geq 1 - \sum_k a_k$:

$$\prod_{j=1}^{n-1} (1 - 2^{-j}) = \frac{1}{2} \prod_{j=2}^{n-1} (1 - 2^{-j}) \geq \frac{1}{2} \left(1 - \sum_{j=2}^{\infty} 2^{-j} \right) = \frac{1}{2} \cdot \frac{1}{2} = \frac{1}{4},$$

con desigualdad estricta para $n \geq 3$ (para $n = 2$ vale $\frac{1}{2}$).

Apartado 2. Mientras la dimensión es r , el número de ejecuciones hasta aumentarla es una variable geométrica de parámetro p_r , cuya media es $1/p_r$. Por la linealidad de la esperanza, el número medio total es

$$\sum_{r=0}^{n-2} \frac{1}{p_r} = \sum_{j=1}^{n-1} \frac{1}{1 - 2^{-j}} = \sum_{j=1}^{n-1} \left(1 + \frac{1}{2^j - 1} \right) = (n - 1) + \sum_{j=1}^{n-1} \frac{1}{2^j - 1}.$$

Como $2^j - 1 \geq 2^{j-1}$, la última suma es menor que $\sum_{j \geq 1} 2^{-(j-1)} = 2$, y el total es menor que $n + 1$. (Numéricamente, $\sum_{j \geq 1} \frac{1}{2^j - 1} \approx 1,607$.) $\square$

Ejemplo: los números para $n = 3$

V tiene 4 elementos. Con $r = 0$ (aún no hay nada útil), cualquier resultado distinto de 000 sirve: $p_0 = \frac{3}{4}$. Con $r = 1$ (W tiene 2 elementos), sirven 2 de los 4: $p_1 = \frac{1}{2}$. La probabilidad de terminar en 2 ejecuciones es $\frac{3}{4} \cdot \frac{1}{2} = \frac{3}{8} = 0,375$, y la media es $\frac{4}{3} + 2 = \frac{10}{3} \approx 3,33$. Una simulación de 20 000 repeticiones da una media de 3,34.

Nota Aclaratoria: el caso $\mathbf{c} = \mathbf{0}$ y el algoritmo completo

Si no se sabe de antemano que $\mathbf{c} \neq \mathbf{0}$, se procede así: se ejecuta el circuito hasta que el rango de los resultados sea al menos $n - 1$. Si llega a n , el sistema solo tiene la solución $\mathbf{0}$ y $\mathbf{c} = \mathbf{0}$. Si es $n - 1$, hay una única solución no nula $\mathbf{c}'$, y con dos consultas clásicas se comprueba si $f(\mathbf{0}) = f(\mathbf{c}')$: si es así, $\mathbf{c} = \mathbf{c}'$; si no, $\mathbf{c} = \mathbf{0}$. En total se usan $O(n)$ consultas y $O(n^3)$ operaciones clásicas (la eliminación gaussiana).

3.3. Implementación en Qiskit

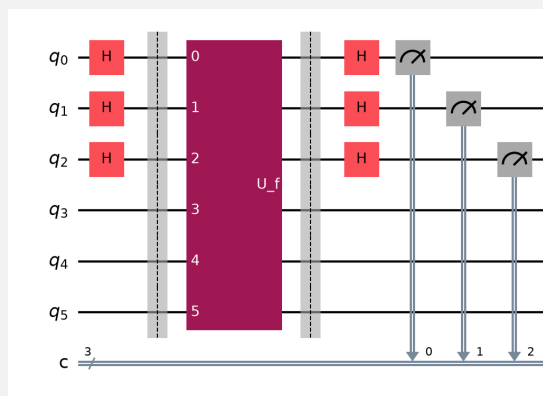
3.3.1. Experimento 4: el algoritmo de Simon con las funciones del libro

Usamos la función `oraculo(f, n, m)` del capítulo anterior con $m = n = 3$, y una función auxiliar que convierte la tabla de valores del libro en una función de enteros:

```
def de_tabla(tabla):
    """Convierte un diccionario {'01': '1', ...} en una función entera
    """
    return lambda x: int(tabla[format(x, "0%db" % len(next(iter(tabla)
    )))], 2)

def simon(f, n):
    qc = QuantumCircuit(2 * n, n)
    qc.h(range(n))
    qc.barrier()
    qc.append(oraculo(f, n, n), range(2 * n))
    qc.barrier()
    qc.h(range(n))
    qc.measure(range(n), range(n))
    return qc

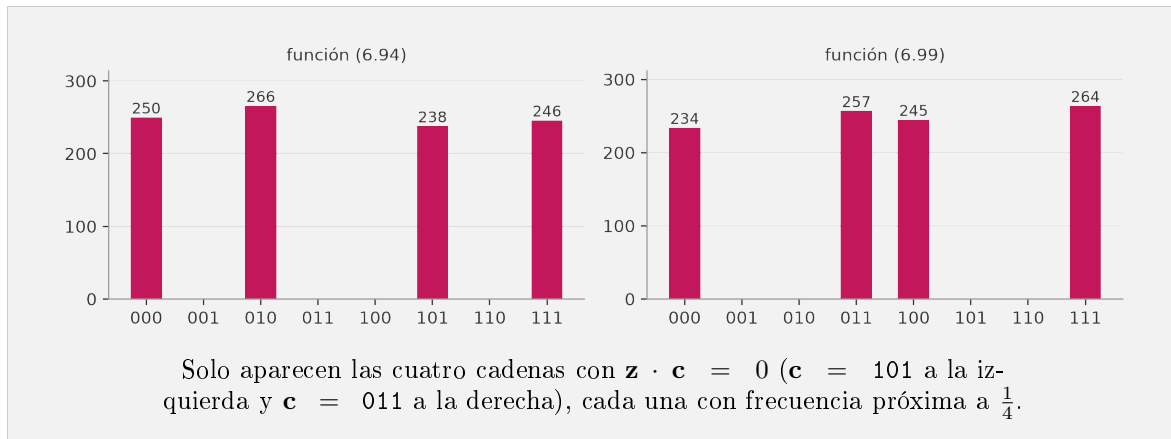
f_694 = {"000": "100", "001": "001", "010": "101", "011": "111",
        "100": "001", "101": "100", "110": "111", "111": "101"}
```



El circuito de Simon con $n = 3$: el registro x son q_0, q_1, q_2 (arriba) y el registro de salida q_3, q_4, q_5 (abajo); solo se mide el de arriba.

Con 1000 disparos:

```
Simon (6.94) {'111': 246, '101': 238, '000': 250, '010': 266}
Simon (6.99) {'011': 257, '111': 264, '100': 245, '000': 234}
```



3.3.2. Experimento 5: el algoritmo completo, ejecución a ejecución

Para resolver el sistema programamos la eliminación gaussiana sobre $\mathbb{F}_2$, guardando cada cadena como un entero para que “sumar ecuaciones” sea el XOR $\wedge$:

```
def resolver_F2(ecuaciones, n):
    """Rango y soluciones  $c \neq 0$  de  $z \cdot c = 0 \pmod{2}$ , por eliminación
    gaussiana."""
    filas, pivotes = [], []
    for z in ecuaciones:
        v = int(z, 2)
        for fila, p in zip(filas, pivotes): # reducir con las filas
            ya guardadas
            if v >> p & 1:
                v ^= fila
        if v: # nueva fila
            independiente
            p = v.bit_length() - 1
            for i in range(len(filas)): # forma escalonada
                reducida
                if filas[i] >> p & 1:
                    filas[i] ^= v
            filas.append(v)
            pivotes.append(p)
    ... # se recorren los valores de las variables libres (ver el
    programa)
    return len(filas), soluciones
```

Ejecutamos el circuito de disparo en disparo (`get_bitstrings()` da el resultado de cada disparo en orden) hasta que el rango es $n - 1 = 2$:

```
(6.94): ejecuciones ['000', '111', '101'] -> rango 2 -> c = ['101']
(6.99): ejecuciones ['111', '111', '111', '100'] -> rango 2 -> c =
['011']
```

En el primer caso el resultado 000 no aporta nada y hacen falta 3 ejecuciones; en el segundo, 111 sale tres veces seguidas y hacen falta 4. Las dos cifras están cerca de la media teórica $\frac{10}{3}$. Por último, el mismo programa resuelve los sistemas del libro:

```
Ejemplo 6.3.2: (6, ['1101010'])
Ejercicio 6.3.3: (5, ['10011001', '01011010', '11000011', '00111100',
                    '10100101', '01100110', '11111111'])
```

El Ejemplo 6.3.2 tiene rango $6 = n - 1$ y solución única; el Ejercicio 6.3.3 tiene rango 5 y siete soluciones no nulas, como se explicó en la corrección.

3.4. Resumen: un ejemplo guiado

Seguimos la función de [Y 6.94], con período $\mathbf{c} = 101$.

Paso 1. El problema

$f(000) = f(101) = 100$, $f(001) = f(100) = 001$, $f(010) = f(111) = 101$,
 $f(011) = f(110) = 111$.

Problema de Simon: $f(\mathbf{x}) = f(\mathbf{y}) \iff \mathbf{y} \in \{\mathbf{x}, \mathbf{x} \oplus \mathbf{c}\}$. Si $\mathbf{c} \neq \mathbf{0}$, f es **dos a uno**.

Paso 2. Clásicamente

Si encontramos $f(000) = f(101)$, entonces $\mathbf{c} = 000 \oplus 101 = 101$. Para garantizarlo hacen falta $2^{n-1} + 1 = 5$ consultas.

Colisión $\Rightarrow$ período. Clásicamente: $2^{n-1} + 1$ (determinista), del orden de $2^{n/2}$ (aleatorio).

Paso 3. Superposición y consulta

$|\varphi_2\rangle = \frac{1}{\sqrt{8}} \sum_{\mathbf{x}} |f(\mathbf{x})\rangle |\mathbf{x}\rangle$: cada valor $\mathbf{w}$ aparece junto a sus dos preimágenes.

El oráculo de $n + n$ qubits **entrelaza** la entrada con la salida. Aquí no hay retroceso de fase: la salida empieza en $|\mathbf{0}\rangle$.

Paso 4. Interferencia

Coefficiente de $|101\rangle |001\rangle$: $\frac{1}{8}((-1)^{010 \cdot 001} + (-1)^{111 \cdot 001}) = \frac{1}{8}(1 - 1) = 0$.

Las preimágenes $\mathbf{x}$ y $\mathbf{x} \oplus \mathbf{c}$ contribuyen con $(-1)^{\mathbf{x} \cdot \mathbf{z}}(1 + (-1)^{\mathbf{c} \cdot \mathbf{z}})$: se cancelan si $\mathbf{z} \cdot \mathbf{c} = 1$.

Paso 5. Medir

Se obtiene 000, 010, 101 o 111, cada uno con probabilidad $\frac{1}{4}$. Qiskit: {'111': 246, '101': 238, '000': 250, '010': 266}.

Cada ejecución da un $\mathbf{z}$ **uniforme** en $\{\mathbf{z} : \mathbf{z} \cdot \mathbf{c} = 0\}$, un subespacio de dimensión $n - 1$.

Paso 6. Ecuaciones

$010 \cdot \mathbf{c} = c_1 = 0$, $101 \cdot \mathbf{c} = c_2 \oplus c_0 = 0 \Rightarrow \mathbf{c} \in \{000, 101\} \Rightarrow \mathbf{c} = 101$.

Eliminación gaussiana sobre $\mathbb{F}_2$. Con rango $n - 1$ las soluciones son exactamente $\{\mathbf{0}, \mathbf{c}\}$ (teorema del rango).

Paso 7. Cuántas veces

Ejecuciones de Qiskit: 000 (inútil), 111, 101: rango 2 tras 3 ejecuciones. Media teórica para $n = 3$: $\frac{10}{3}$.

Número medio de ejecuciones $< n + 1$; probabilidad de terminar en $n - 1$ ejecuciones $> \frac{1}{4}$. (Yanofsky dice “ n evaluaciones”: es la idea, no un número exacto.)

Paso	Qué hicimos	Concepto
1	Tabla de f	Período $\mathbf{c}$; funciones dos a uno
2	Buscar una colisión	Cota clásica; paradoja del cumpleaños
3	U_f sobre la superposición	Entrelazamiento entrada-salida
4	Un coeficiente a mano	Interferencia destructiva si $\mathbf{z} \cdot \mathbf{c} = 1$
5	Medir	$\mathbf{z}$ uniforme en $\mathbf{c}^\perp$
6	Resolver	Álgebra lineal sobre $\mathbb{F}_2$
7	Contar ejecuciones	Media $< n + 1$

Capítulo 4

El algoritmo de búsqueda de Grover

El algoritmo de Grover [6] busca un elemento marcado entre $N = 2^n$ posibles con unas $\frac{\pi}{4}\sqrt{N}$ consultas, frente a las del orden de N que necesita cualquier algoritmo clásico. La ventaja es “solo” cuadrática, pero el problema es mucho más general que los de los capítulos anteriores (no hay ninguna promesa sobre la estructura de f) y se sabe que ningún algoritmo cuántico puede hacerlo mejor [8]. Combina dos reflexiones: la **inversión de fase** y la **inversión respecto de la media**. Yanofsky deja sin demostrar por qué hay que repetirlas unas $\sqrt{N}$ veces; aquí se demuestra, con la interpretación geométrica que el libro recomienda. Corresponde a la sección 6.4 de Yanofsky.

4.1. El problema y la solución clásica

El problema de búsqueda

Se da $f : \{0, 1\}^n \rightarrow \{0, 1\}$ como caja negra, con la promesa de que hay **exactamente una** cadena $\mathbf{x}_0$ con $f(\mathbf{x}_0) = 1$ [Y 6.100]:

$$f(\mathbf{x}) = \begin{cases} 1 & \text{si } \mathbf{x} = \mathbf{x}_0, \\ 0 & \text{si } \mathbf{x} \neq \mathbf{x}_0. \end{cases}$$

Hay que encontrar $\mathbf{x}_0$. Escribimos $N = 2^n$ para el número de candidatos.

Es la versión abstracta de buscar en una lista desordenada: $f(\mathbf{x})$ responde a la pregunta “¿es $\mathbf{x}$ el elemento buscado?”.

Proposición: cota clásica

Un algoritmo clásico determinista necesita $N - 1$ consultas en el peor caso. Uno aleatorio que acierte con probabilidad al menos $\frac{1}{2}$ necesita del orden de N consultas (al menos $N/2 - 1$).

Demostración

Determinista. Con $N - 1$ consultas basta: si todas dan 0, el elemento es el que falta. Si se hacen $k \leq N - 2$ consultas, un adversario responde 0 a todas; quedan al menos dos candidatos no consultados, ambos compatibles con todo lo observado, y el algoritmo falla en uno de ellos.

Aleatorio. Supongamos que $\mathbf{x}_0$ se elige uniformemente al azar. Mientras no se encuentra $\mathbf{x}_0$, todas las respuestas son 0, así que el algoritmo no aprende nada salvo qué cadenas descartar. La probabilidad de que $\mathbf{x}_0$ esté entre las k cadenas consultadas es k/N ; si no está, el algoritmo tiene que adivinar entre las $N - k$ restantes y acierta con probabilidad $1/(N - k)$. En total,

$$\Pr(\text{éxito}) \leq \frac{k}{N} + \frac{N - k}{N} \cdot \frac{1}{N - k} = \frac{k + 1}{N},$$

y para que valga al menos $\frac{1}{2}$ hace falta $k \geq \frac{N}{2} - 1$. (Si funciona con probabilidad $\geq \frac{1}{2}$ para todo $\mathbf{x}_0$, también la tiene en media sobre $\mathbf{x}_0$ elegido al azar.) $\square$

En media, un algoritmo que consulta en orden aleatorio encuentra $\mathbf{x}_0$ tras unas $N/2$ consultas [Y p. 196]. Grover lo hace con unas $\frac{\pi}{4}\sqrt{N}$: para $N = 10^6$, unas 785 consultas en lugar de medio millón.

4.2. El algoritmo cuántico

4.2.1. El oráculo y un primer intento

El oráculo es el mismo de Deutsch–Jozsa, $U_f |y\rangle |\mathbf{x}\rangle = |y \oplus f(\mathbf{x})\rangle |\mathbf{x}\rangle$: solo cambia y cuando $\mathbf{x} = \mathbf{x}_0$.

Ejercicio 6.4.1 (Yanofsky): los oráculos de búsqueda para $n = 2$

En el orden de Qiskit (base $|y x_1 x_0\rangle$), el oráculo que marca $\mathbf{x}_0$ intercambia $|0\rangle |\mathbf{x}_0\rangle \leftrightarrow |1\rangle |\mathbf{x}_0\rangle$ y deja fijos los otros seis vectores. Es decir, en la matriz 8×8 (identidad salvo un intercambio de dos columnas):

$\mathbf{x}_0$	vectores que se intercambian
00	$ 000\rangle \leftrightarrow 100\rangle$
01	$ 001\rangle \leftrightarrow 101\rangle$
10	$ 010\rangle \leftrightarrow 110\rangle$ (el de [Y 6.101])
11	$ 011\rangle \leftrightarrow 111\rangle$

En [Y 6.101], con el orden de Yanofsky, el intercambio es $|10, 0\rangle \leftrightarrow |10, 1\rangle$: los mismos dos estados. Cada uno de estos oráculos es una puerta X sobre y **controlada** por los dos qubits de $\mathbf{x}$, con los controles “en 0” donde $\mathbf{x}_0$ tiene un 0 (Experimento 7).

Primer intento [Y 6.102]: poner $\mathbf{x}$ en superposición uniforme, y en $|0\rangle$, y consultar. Se obtiene $|\varphi_2\rangle = \frac{1}{\sqrt{N}} \sum_{\mathbf{x}} |f(\mathbf{x})\rangle |\mathbf{x}\rangle$. Al medir, $y = 1$ con probabilidad $\frac{1}{N}$ (solo la componente $\mathbf{x}_0$ tiene $y = 1$), y en ese caso el registro de arriba da $\mathbf{x}_0$. Pero eso es tan improbable como acertar al azar. Hace falta amplificar la amplitud de $\mathbf{x}_0$.

4.2.2. Primer ingrediente: la inversión de fase

Con y en $|-\rangle$, el lema del retroceso de fase da $U_f(|-\rangle |\mathbf{x}\rangle) = (-1)^{f(\mathbf{x})} |-\rangle |\mathbf{x}\rangle$ [Y 6.113]. Sobre el registro de arriba, el efecto es el operador diagonal

$$Z_f = \sum_{\mathbf{x}} (-1)^{f(\mathbf{x})} |\mathbf{x}\rangle \langle \mathbf{x}| = I - 2 |\mathbf{x}_0\rangle \langle \mathbf{x}_0|,$$

que **cambia el signo de la amplitud de $\mathbf{x}_0$** y deja las demás igual. Es la **inversión de fase**.

Ejemplo: inversión de fase con $n = 2$

Con $\mathbf{x}_0 = 10$ y el estado uniforme $\frac{1}{2}(1, 1, 1, 1)^T$ (componentes 00, 01, 10, 11), la inversión de fase da $\frac{1}{2}(1, 1, -1, 1)^T$ [Y p. 198]. Medir no distingue nada: $|\pm \frac{1}{2}|^2 = \frac{1}{4}$ para las cuatro cadenas. La fase separa $\mathbf{x}_0$ de las demás, pero no de forma observable.

Nota Aclaratoria: el qubit auxiliar se prepara una sola vez

Yanofsky escribe la inversión de fase como $U_f(I \otimes H)$ aplicada a $|\mathbf{x}, 1\rangle$ [Y 6.109], e incluye esa H dentro del bucle del algoritmo [Y 6.134]. Leído al pie de la letra, aplicar H en cada iteración devolvería y de $|-\rangle$ a $|1\rangle$ y estropearía el algoritmo. Lo correcto es preparar $|-\rangle = H|1\rangle$ **una vez** al principio: por el lema del retroceso de fase, $|-\rangle$ es un autovector de U_f y **sigue siendo $|-\rangle$ tras cada consulta**, así que cada aplicación posterior de U_f actúa como Z_f sobre el registro de arriba.

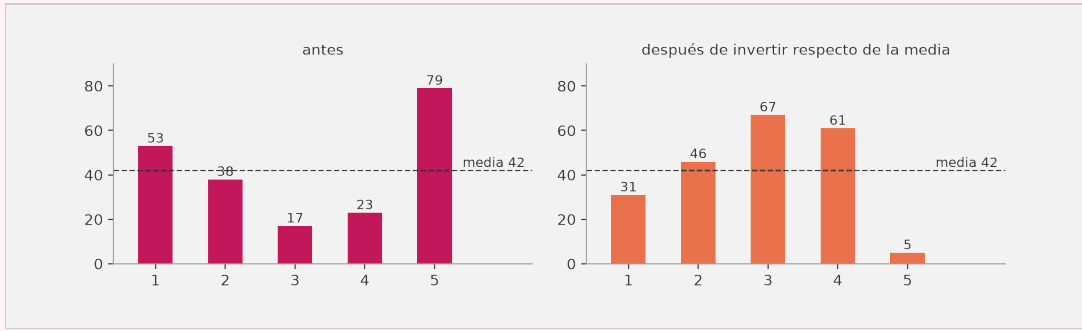
4.2.3. Segundo ingrediente: la inversión respecto de la media

Dada una lista de números $v_1, \dots, v_N$ con media $a = \frac{1}{N} \sum_i v_i$, **invertir respecto de la media** es reflejar cada número al otro lado de a , a la misma distancia [Y 6.114, 6.115]:

$$v'_i = a + (a - v_i) = 2a - v_i.$$

Ejemplo: la lista 53, 38, 17, 23, 79 (figuras 6.1 y 6.2 del libro)

La media es $a = \frac{210}{5} = 42$. Entonces $v'_1 = 84 - 53 = 31$, $v'_2 = 84 - 38 = 46$, $v'_3 = 67$, $v'_4 = 61$, $v'_5 = 5$. La media de la lista nueva sigue siendo $\frac{31+46+67+61+5}{5} = 42$: los números que estaban 11 por encima quedan 11 por debajo, y al revés.



Ejercicio 6.4.2 (Yanofsky)

Para 5, 38, 62, 58, 21, 35: la media es $\frac{219}{6} = 36,5$ y $2a = 73$, así que la lista invertida es $73-5 = 68$, $73-38 = 35$, $73-62 = 11$, $73-58 = 15$, $73-21 = 52$, $73-35 = 38$, de media $\frac{219}{6} = 36,5$ otra vez.

En forma matricial, la media se obtiene con la matriz A de tamaño $N \times N$ con todas las entradas iguales a $\frac{1}{N}$ [Y 6.116, 6.122]: AV es el vector con todas las componentes iguales a la media de V . La inversión es entonces $V' = 2AV - V = (-I + 2A)V$ [Y 6.118]. Para la lista anterior, $(-I + 2A)(5, 38, 62, 58, 21, 35)^T = (68, 35, 11, 15, 52, 38)^T$ [Y 6.121], comprobado con `numpy`.

Proposición: la inversión respecto de la media es una reflexión unitaria

Sea $N = 2^n$, A la matriz $N \times N$ con todas las entradas $\frac{1}{N}$, $D = -I + 2A$ y $|s\rangle = H^{\otimes n} |0\rangle = \frac{1}{\sqrt{N}} \sum_{\mathbf{x}} |\mathbf{x}\rangle$. Entonces:

1. $A^2 = A$ (Ejercicio 6.4.3), y de hecho $A = |s\rangle \langle s|$;
2. D es unitaria y $D^2 = I$;
3. $D = 2|s\rangle \langle s| - I = H^{\otimes n}(2|0\rangle \langle 0| - I)H^{\otimes n}$.

Geoméricamente, D es la **reflexión respecto de la recta generada por $|s\rangle$** : deja fijo $|s\rangle$ y cambia el signo de todo vector ortogonal a $|s\rangle$.

Demostración

1. La entrada $(\mathbf{x}, \mathbf{y})$ de $|s\rangle\langle s|$ es $\langle \mathbf{x}|s\rangle\langle s|\mathbf{y}\rangle = \frac{1}{\sqrt{N}} \cdot \frac{1}{\sqrt{N}} = \frac{1}{N}$, así que $A = |s\rangle\langle s|$. Como $\langle s|s\rangle = N \cdot \frac{1}{N} = 1$, $A^2 = |s\rangle\langle s|s\rangle\langle s| = |s\rangle\langle s| = A$. (Directamente: la entrada $(\mathbf{x}, \mathbf{y})$ de A^2 es $\sum_{\mathbf{z}} \frac{1}{N} \cdot \frac{1}{N} = N \cdot \frac{1}{N^2} = \frac{1}{N}$.)
2. D es real y simétrica, luego $D^\dagger = D$, y [Y 6.124]

$$D^\dagger D = D^2 = (-I + 2A)^2 = I - 4A + 4A^2 = I - 4A + 4A = I.$$

3. La primera igualdad es $D = -I + 2A$ con $A = |s\rangle\langle s|$. Para la segunda, como $H^{\otimes n}$ es autoadjunta, $(H^{\otimes n})^2 = I$ y $H^{\otimes n}|\mathbf{0}\rangle = |s\rangle$:

$$H^{\otimes n}(2|\mathbf{0}\rangle\langle \mathbf{0}| - I)H^{\otimes n} = 2(H^{\otimes n}|\mathbf{0}\rangle)(H^{\otimes n}|\mathbf{0}\rangle)^\dagger - (H^{\otimes n})^2 = 2|s\rangle\langle s| - I.$$

Reflexión. $D|s\rangle = 2|s\rangle\langle s|s\rangle - |s\rangle = |s\rangle$, y si $\langle s|\psi\rangle = 0$, $D|\psi\rangle = -|\psi\rangle$. $\square$

La inversión de fase también es una reflexión: $Z_f = I - 2|\mathbf{x}_0\rangle\langle \mathbf{x}_0|$ deja fijo todo vector ortogonal a $|\mathbf{x}_0\rangle$ y cambia el signo de $|\mathbf{x}_0\rangle$. Grover alterna dos reflexiones, y la composición de dos reflexiones es un **giro**. Esta es la clave de la demostración de la sección siguiente.

Ejemplo 6.4.1 (Yanofsky) y Ejercicio 6.4.4: las dos operaciones juntas

Partimos de $(10, 10, 10, 10, 10)$ y marcamos siempre la cuarta componente (el vector no está normalizado; lo que importa es el mecanismo). Cada fila es una iteración: primero se cambia el signo de la cuarta componente y después se invierte respecto de la media.

k	media a	componentes 1, 2, 3, 5	componente 4 (marcada)	diferencia
1	6	2	22	20
2	-2,8	-7,6	16,4	24
3	-9,36	-11,12	-2,32	8,8

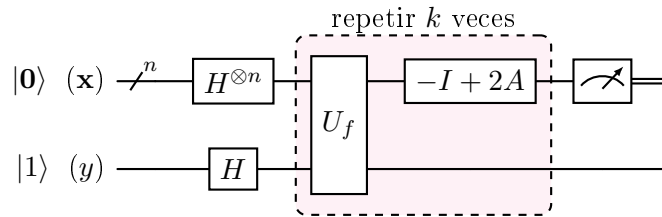
Por ejemplo, en la fila 2 la lista tras la inversión de fase es $(2, 2, 2, -22, 2)$, de media $\frac{8-22}{5} = -2,8$. La fila 3 es el Ejercicio 6.4.4: **el resultado empeora**. La cuarta componente, que llevaba ventaja, queda por debajo de las demás en valor absoluto ($|-2,32| < |-11,12|$). Es lo que Yanofsky llama “pasarse de cocción” (*overcook*). Observemos también que la suma de los cuadrados es 500 en todas las filas: las dos operaciones conservan la norma, como debe ser para matrices unitarias.

4.2.4. El algoritmo

Llamamos **iteración de Grover** a $G = DZ_f$ (primero la inversión de fase, después la inversión respecto de la media).

Algoritmo de Grover (versión corregida del enunciado del libro)

1. Preparar el registro de n qubits en $|0\rangle$ y el qubit auxiliar en $|1\rangle$.
2. Aplicar $H^{\otimes n}$ al registro (queda $|s\rangle$) y H al auxiliar (queda $|-\rangle$), **una sola vez**.
3. Repetir $k = \lfloor \pi/(4\theta) \rfloor$ veces, donde $\theta = \arcsin(1/\sqrt{N})$ (para N grande, $k \approx \frac{\pi}{4}\sqrt{N}$):
 - 3a. inversión de fase: aplicar U_f (que actúa como Z_f sobre el registro);
 - 3b. inversión respecto de la media: aplicar $D = -I + 2A$ al registro.
4. Medir el registro.



Una corrección al libro: el número de iteraciones

Yanofsky dice que hay que repetir el paso 3 “ $\sqrt{2^n}$ veces” [Y p. 202]. El orden de magnitud es correcto, pero **el número exacto importa**, porque el algoritmo es periódico y pasarse lo estropea:

- para $n = 3$, $\sqrt{8} \approx 2,83$, que redondeado da 3 iteraciones y una probabilidad de éxito de solo 0,330; con $k = \lfloor \pi/(4\theta) \rfloor = 2$ iteraciones (las que hace el propio ejemplo del libro) es 0,945;
- para $n = 4$, $\sqrt{16} = 4$ iteraciones dan 0,582, mientras que $k = 3$ da 0,961.

El número correcto es aproximadamente $\frac{\pi}{4}\sqrt{2^n} \approx 0,785\sqrt{2^n}$, y se demuestra en el teorema siguiente.

Ejemplo 6.4.2 (Yanofsky): $n = 3$, $\mathbf{x}_0 = 101$

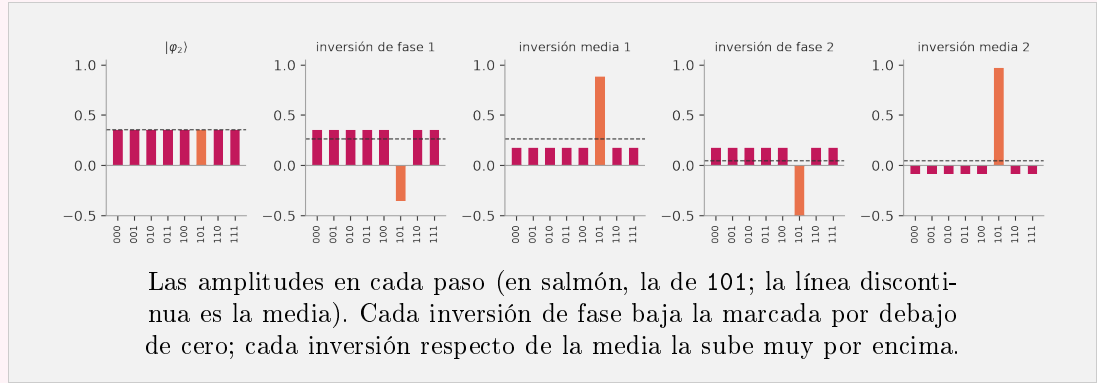
Escribimos las amplitudes del registro (componentes 000 a 111; la de 101 es la sexta).

Superposición. $|\varphi_2\rangle = \frac{1}{\sqrt{8}}(1, 1, 1, 1, 1, 1, 1, 1)$ [Y 6.136].

Iteración 1. Inversión de fase: $\frac{1}{\sqrt{8}}(1, 1, 1, 1, 1, -1, 1, 1)$. Media: $a = \frac{1}{8} \cdot \frac{7-1}{\sqrt{8}} = \frac{3}{4\sqrt{8}}$ [Y 6.138]. Inversión: las no marcadas pasan a $2a - \frac{1}{\sqrt{8}} = \frac{3}{2\sqrt{8}} - \frac{2}{2\sqrt{8}} = \frac{1}{2\sqrt{8}}$, y la marcada a $2a + \frac{1}{\sqrt{8}} = \frac{5}{2\sqrt{8}}$ [Y 6.139–6.141].

Iteración 2. Inversión de fase: la sexta pasa a $-\frac{5}{2\sqrt{8}}$. Media: $a = \frac{1}{8} \cdot \frac{7-5}{2\sqrt{8}} = \frac{1}{8\sqrt{8}}$ [Y 6.143]. Inversión: no marcadas $\frac{2}{8\sqrt{8}} - \frac{1}{2\sqrt{8}} = -\frac{1}{4\sqrt{8}}$; marcada $\frac{2}{8\sqrt{8}} + \frac{5}{2\sqrt{8}} = \frac{11}{4\sqrt{8}}$ [Y 6.144–6.146].

Medición. $\text{Pr}(101) = \left(\frac{11}{4\sqrt{8}}\right)^2 = \frac{121}{128} \approx 0,9453$ y cada una de las otras siete sale con probabilidad $\frac{1}{128}$ ($7 \cdot \frac{1}{128} + \frac{121}{128} = 1$). Los decimales del libro son $\frac{11}{4\sqrt{8}} \approx 0,97227$ y $-\frac{1}{4\sqrt{8}} \approx -0,08839$.

**4.2.5. Demostración: por qué $\frac{\pi}{4}\sqrt{N}$ iteraciones**

Yanofsky afirma que la prueba “está fuera del alcance del texto” pero usa “una geometría muy bonita”. Es esta.

Teorema: el algoritmo de Grover gira el estado

Sean $|\beta\rangle = |\mathbf{x}_0\rangle$, $|\alpha\rangle = \frac{1}{\sqrt{N-1}} \sum_{\mathbf{x} \neq \mathbf{x}_0} |\mathbf{x}\rangle$ y $\theta \in (0, \frac{\pi}{2}]$ con $\sin \theta = \frac{1}{\sqrt{N}}$. Entonces, para todo $k \geq 0$,

$$G^k |s\rangle = \cos((2k+1)\theta) |\alpha\rangle + \sin((2k+1)\theta) |\beta\rangle,$$

y la probabilidad de obtener $\mathbf{x}_0$ tras k iteraciones es $\sin^2((2k+1)\theta)$.

Demostración

El plano invariante. $|\alpha\rangle$ y $|\beta\rangle$ son ortonormales (tienen soporte disjunto y norma 1), y

$$|s\rangle = \frac{1}{\sqrt{N}} \sum_{\mathbf{x}} |\mathbf{x}\rangle = \sqrt{\frac{N-1}{N}} |\alpha\rangle + \frac{1}{\sqrt{N}} |\beta\rangle = \cos \theta |\alpha\rangle + \sin \theta |\beta\rangle,$$

pues $\cos \theta = \sqrt{1 - 1/N}$. Veamos que Z_f y D envían el plano $P = \text{span}\{|\alpha\rangle, |\beta\rangle\}$ a sí mismo y calculemos sus matrices en la base $(|\alpha\rangle, |\beta\rangle)$.

Inversión de fase. $Z_f |\alpha\rangle = |\alpha\rangle$ (no contiene $\mathbf{x}_0$) y $Z_f |\beta\rangle = -|\beta\rangle$: $Z_f|_P = \begin{pmatrix} 1 & 0 \\ 0 & -1 \end{pmatrix}$, la reflexión respecto del eje $|\alpha\rangle$.

Inversión respecto de la media. $D = 2|s\rangle\langle s| - I$, y $|s\rangle = (\cos \theta, \sin \theta)^T$ en esta base. Para $|v\rangle \in P$, $D|v\rangle = 2\langle s|v\rangle |s\rangle - |v\rangle \in P$, con matriz

$$D|_P = 2 \begin{pmatrix} \cos \theta \\ \sin \theta \end{pmatrix} (\cos \theta \quad \sin \theta) - I = \begin{pmatrix} 2 \cos^2 \theta - 1 & 2 \sin \theta \cos \theta \\ 2 \sin \theta \cos \theta & 2 \sin^2 \theta - 1 \end{pmatrix} = \begin{pmatrix} \cos 2\theta & \sin 2\theta \\ \sin 2\theta & -\cos 2\theta \end{pmatrix}.$$

La iteración es un giro. Multiplicando,

$$G|_P = D|_P Z_f|_P = \begin{pmatrix} \cos 2\theta & \sin 2\theta \\ \sin 2\theta & -\cos 2\theta \end{pmatrix} \begin{pmatrix} 1 & 0 \\ 0 & -1 \end{pmatrix} = \begin{pmatrix} \cos 2\theta & -\sin 2\theta \\ \sin 2\theta & \cos 2\theta \end{pmatrix},$$

que es el **giro de ángulo** 2θ . Por tanto $G^k|_P$ es el giro de ángulo $2k\theta$ (la potencia de un giro es el giro del ángulo multiplicado, por las fórmulas de adición del seno y el coseno, o por inducción), y aplicado a $|s\rangle$, que forma un ángulo θ con $|\alpha\rangle$, da el vector de ángulo $(2k+1)\theta$:

$$G^k |s\rangle = \cos((2k+1)\theta) |\alpha\rangle + \sin((2k+1)\theta) |\beta\rangle.$$

Probabilidad. La amplitud de $|\mathbf{x}_0\rangle = |\beta\rangle$ es $\sin((2k+1)\theta)$, y el qubit auxiliar está en $|-\rangle$, factorizado. Por la regla de Born, $\Pr(\mathbf{x}_0) = \sin^2((2k+1)\theta)$. $\square$

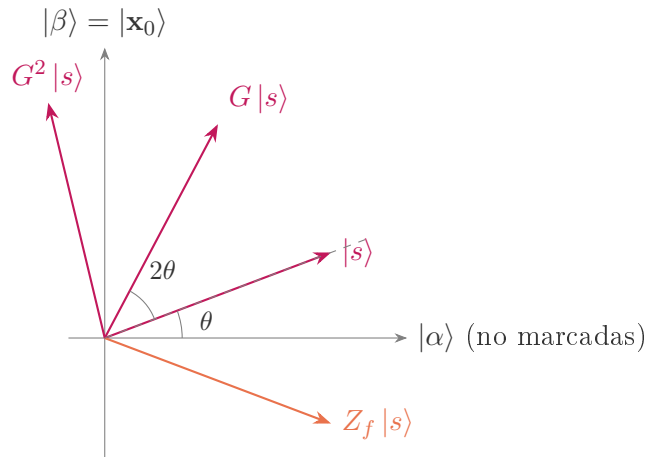


Figura: el caso $N = 8$ ($\theta \approx 20,7^\circ$). Z_f refleja $|s\rangle$ respecto del eje horizontal; D refleja el resultado respecto de la recta de $|s\rangle$ (discontinua). El efecto neto es un giro de 2θ hacia $|\mathbf{x}_0\rangle$. Tras dos iteraciones el ángulo es $5\theta \approx 103,5^\circ$, ya muy cerca de la vertical: $\sin^2(5\theta) = 0,945$.

Corolario: número de iteraciones

Con $k = \lfloor \pi/(4\theta) \rfloor$ iteraciones, la probabilidad de éxito es al menos $1 - \frac{1}{N}$, y $k \leq \frac{\pi}{4} \sqrt{N}$.

Demostración

De $\frac{\pi}{4\theta} - 1 < k \leq \frac{\pi}{4\theta}$ se obtiene, multiplicando por 2θ y sumando θ , $\frac{\pi}{2} - \theta < (2k+1)\theta \leq \frac{\pi}{2} + \theta$. Es decir, el ángulo final dista de $\frac{\pi}{2}$ como mucho θ , y como $\sin^2$ es simétrica respecto de $\frac{\pi}{2}$ y creciente en $[0, \frac{\pi}{2}]$,

$$\sin^2((2k+1)\theta) \geq \sin^2\left(\frac{\pi}{2} - \theta\right) = \cos^2\theta = 1 - \frac{1}{N}.$$

Además, $\theta \geq \sin\theta = \frac{1}{\sqrt{N}}$ (pues $\sin t \leq t$ para $t \geq 0$), luego $k \leq \frac{\pi}{4\theta} \leq \frac{\pi}{4} \sqrt{N}$. $\square$

Ejemplo: los números para $n = 3$ y $n = 4$

$n = 3$: $\theta = \arcsin(1/\sqrt{8}) \approx 0,3614$, $\frac{\pi}{4\theta} \approx 2,17$, $k = 2$ y $\text{Pr} = \sin^2(5\theta) = \frac{121}{128} \approx 0,945 \geq 1 - \frac{1}{8}$.

$n = 4$: $\theta = \arcsin(\frac{1}{4}) \approx 0,2527$, $\frac{\pi}{4\theta} \approx 3,11$, $k = 3$ y $\text{Pr} = \sin^2(7\theta) \approx 0,961 \geq 1 - \frac{1}{16}$.

Con más iteraciones el estado sigue girando y **se aleja** de $|\mathbf{x}_0\rangle$: para $n = 3$, $\text{Pr} = 0,330$ con $k = 3$ y $0,012$ con $k = 4$, y vuelve a acercarse con $k = 6$ ($0,9998$). Esto explica el “exceso de cocción” del Ejercicio 6.4.4.

Ejercicio 6.4.5 (Yanofsky): $n = 4$, $\mathbf{x}_0 = 1101$

Hay 16 amplitudes, todas iguales a $\frac{1}{4}$ al principio. Por simetría, en cada paso las 15 no marcadas son iguales entre sí; llamamos u a su valor y m al de la marcada. Cada iteración hace $m \mapsto -m$ y luego, con $a = \frac{15u-m}{16}$, $u \mapsto 2a - u$ y $-m \mapsto 2a + m$.

k	media a	u	m	$\text{Pr}(1101) = m^2$
0	—	$\frac{1}{4}$	$\frac{1}{4}$	0,0625
1	$\frac{7}{32}$	$\frac{3}{16}$	$\frac{11}{16}$	0,4727
2	$\frac{17}{128}$	$\frac{5}{64}$	$\frac{61}{64}$	0,9084
3	$\frac{7}{512}$	$-\frac{13}{256}$	$\frac{251}{256}$	0,9613
4	$-\frac{223}{2048}$	$-\frac{171}{1024}$	$\frac{781}{1024}$	0,5817

Por ejemplo, en la iteración 1: $a = \frac{15 \cdot \frac{1}{4} - \frac{1}{4}}{16} = \frac{14/4}{16} = \frac{7}{32}$, $u = \frac{7}{16} - \frac{1}{4} = \frac{3}{16}$ y $m = \frac{7}{16} + \frac{1}{4} = \frac{11}{16}$. Comprobación de la norma en $k = 3$: $15 \cdot \frac{169}{65536} + \frac{63001}{65536} = \frac{65536}{65536} = 1$. Lo óptimo son 3 iteraciones, como da el corolario; con las 4 que sugiere el libro la probabilidad cae a 0,58. Los valores de m coinciden con $\sin((2k+1)\theta)$ para $\theta = \arcsin \frac{1}{4}$.

Nota Aclaratoria: varias soluciones y optimalidad

Varias soluciones. Si hay t cadenas marcadas [Y p. 204], la misma demostración vale con $|\beta\rangle$ la superposición uniforme de las marcadas, $|\alpha\rangle$ la de las no marcadas y $\sin\theta = \sqrt{t/N}$. El número de iteraciones es $\lfloor \pi/(4\theta) \rfloor \approx \frac{\pi}{4} \sqrt{N/t}$ (el libro da $\sqrt{N/t}$, que es el orden de magnitud). Un caso curioso: si $t = N/4$, $\theta = \frac{\pi}{6}$ y **una** iteración da probabilidad $\sin^2(\frac{\pi}{2}) = 1$. Si t no se conoce, hay variantes que eligen el número de iteraciones al azar [7].

Optimalidad. Bennett, Bernstein, Brassard y Vazirani [8] demostraron que todo algoritmo cuántico que resuelva el problema de búsqueda necesita del orden de $\sqrt{N}$ consultas: Grover es óptimo. Por eso la ventaja cuántica en búsqueda no estructurada es cuadrática y no exponencial.

Conexión con el TFG

El ángulo inicial del algoritmo es $\sin^2\theta = |\langle s|\mathbf{x}_0\rangle|^2 = \frac{1}{N} = 2^{-n}$: el estado uniforme y un estado de la base son **casi ortogonales** cuando hay muchos qubits. Es el mismo fenómeno que está detrás de la **concentración exponencial** de los kernels cuánticos (Thanasilp et al. [12]): en un espacio de dimensión 2^n , dos estados “típicos” tienen fidelidad $|\langle \phi(\mathbf{x})|\phi(\mathbf{y})\rangle|^2$ del orden de 2^{-n} , y distinguir valores tan pequeños con disparos requiere del orden de 2^n repeticiones. Grover saca partido de amplitudes de tamaño $2^{-n/2}$ amplificándolas coherentemente; un kernel estimado midiendo no puede hacerlo.

4.3. Implementación en Qiskit

4.3.1. Experimento 6: el oráculo y la difusión con puertas

Esta vez construimos las dos reflexiones con puertas, como se haría en un ordenador cuántico real. El oráculo es una X multicontrolada sobre el auxiliar (mcx), con puertas X antes y después sobre los qubits en los que $\mathbf{x}_0$ tiene un 0, para que el control se active con $|\mathbf{x}_0\rangle$ y solo con él:

```
def oraculo_grover(marcado, n):
    qc = QuantumCircuit(n + 1, name="U_f")
    ceros = [q for q in range(n) if not (marcado >> q) & 1]
    if ceros:
        qc.x(ceros)
    qc.mcx(list(range(n)), n)
    if ceros:
        qc.x(ceros)
    return qc
```

Para la difusión usamos la proposición: $D = H^{\otimes n}(2|0\rangle\langle 0| - I)H^{\otimes n}$. A su vez, $I - 2|0\rangle\langle 0| = X^{\otimes n}(I - 2|1\dots 1\rangle\langle 1\dots 1|)X^{\otimes n}$, y $I - 2|1\dots 1\rangle\langle 1\dots 1|$ es la Z multicontrolada, que se obtiene como H , X multicontrolada y H sobre el último qubit. Juntando,

$$H^{\otimes n}X^{\otimes n}(\text{MCZ})X^{\otimes n}H^{\otimes n} = H^{\otimes n}(I - 2|0\rangle\langle 0|)H^{\otimes n} = -D.$$

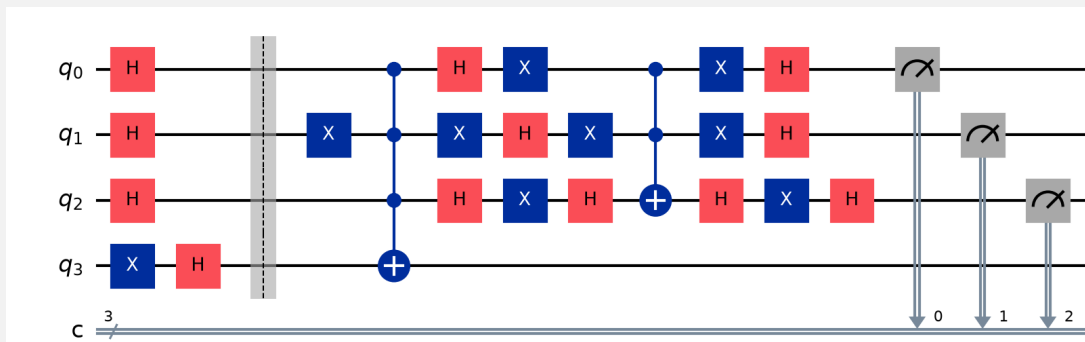
```
def difusion(n):
    """-(H X MCZ X H) = -I + 2A, salvo el signo global."""
    qc = QuantumCircuit(n, name="-I+2A")
    qc.h(range(n))
    qc.x(range(n))
    qc.h(n - 1)
    qc.mcx(list(range(n - 1)), n - 1)
    qc.h(n - 1)
    qc.x(range(n))
    qc.h(range(n))
    return qc
```

El circuito implementa $-D$: difiere de D en una **fase global** -1 , que no afecta a ninguna probabilidad. Lo comprobamos para $n = 2, 3, 4$:

```
n=2: Operator(difusion) == -(-I+2A): True      oráculo con puertas ==
      U_f: True
n=3: Operator(difusion) == -(-I+2A): True      oráculo con puertas ==
      U_f: True
n=4: Operator(difusion) == -(-I+2A): True      oráculo con puertas ==
      U_f: True
```

4.3.2. Experimento 7: el algoritmo completo

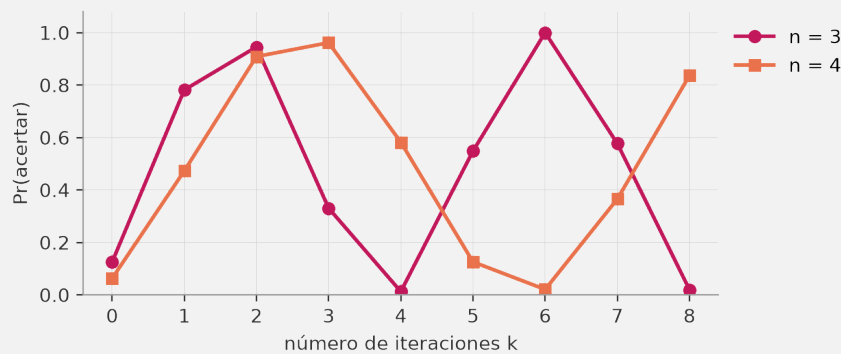
```
def grover(marcado, n, iteraciones, medir=True):
    qc = QuantumCircuit(n + 1, n)
    qc.x(n)
    qc.h(n)
    qc.h(range(n))
    for _ in range(iteraciones):
        qc.barrier()
        qc.compose(oraculo_grover(marcado, n), range(n + 1), inplace=True)
        qc.compose(difusion(n), range(n), inplace=True)
    if medir:
        qc.measure(range(n), range(n))
    return qc
```



Una iteración de Grover para $n = 3$ y $\mathbf{x}_0 = 101$. El oráculo es la X con tres controles sobre q_3 , rodeada de X sobre q_1 (el bit central de 101 es 0); después viene la difusión.

Las probabilidades exactas (con `Statevector`) coinciden con la fórmula $\sin^2((2k + 1)\theta)$ del teorema en todos los casos:

k	0	1	2	3	4	5	6
$n = 3, 101$	0,12500	0,78125	0,94531	0,33008	0,01221	0,54797	0,99979
$n = 4, 1101$	0,06250	0,47266	0,90845	0,96132	0,58170	0,12549	0,02038



Probabilidad de acertar frente al número de iteraciones. Es periódica: el estado gira en el plano $\{|\alpha\rangle, |\beta\rangle\}$. Los máximos están en $k = 2$ para $n = 3$ y en $k = 3$ para $n = 4$.

Las amplitudes del registro tras $k = 1$ y $k = 2$, multiplicadas por $\sqrt{8}$, son

```

k=1: [-0.5  -0.5  -0.5  -0.5  -0.5  -2.5  -0.5  -0.5 ]
k=2: [-0.25 -0.25 -0.25 -0.25 -0.25  2.75 -0.25 -0.25]

```

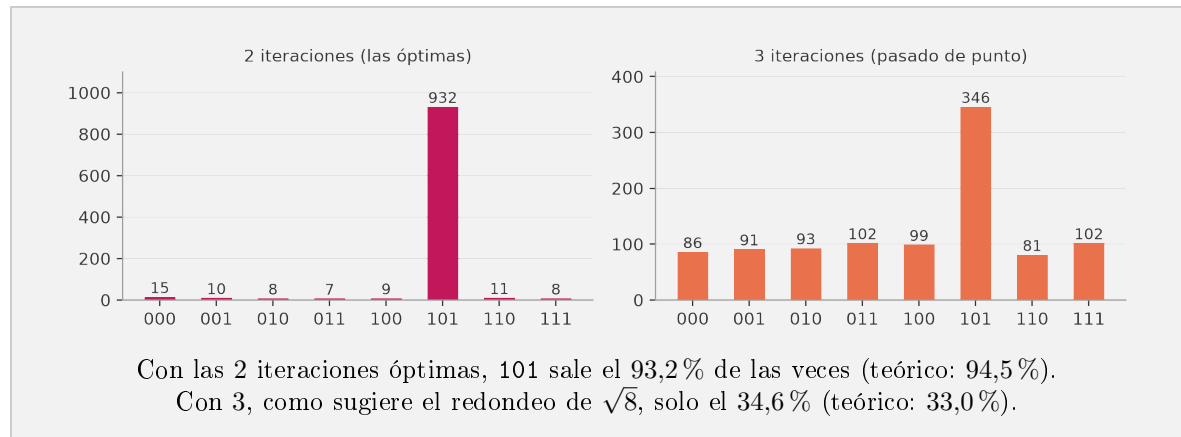
que son las del Ejemplo 6.4.2 ($\frac{1}{2\sqrt{8}}, \frac{5}{2\sqrt{8}}$ y $-\frac{1}{4\sqrt{8}}, \frac{11}{4\sqrt{8}}$) multiplicadas por $(-1)^k$: cada difusión aporta el signo global -1 del Experimento 6.

Finalmente, con 1000 disparos:

```

2 iteraciones: {'101': 932, '110': 11, '011': 7, '100': 9, '000': 15,
                '111': 8, '010': 8, '001': 10}
3 iteraciones: {'011': 102, '100': 99, '101': 346, '001': 91, '110':
                81, '000': 86, '010': 93, '111': 102}

```



4.4. Resumen: un ejemplo guiado

Seguimos el ejemplo del libro: $n = 3$, $N = 8$ y el elemento marcado es $\mathbf{x}_0 = 101$.

Paso 1. El problema

$f(101) = 1$ y $f(\mathbf{x}) = 0$ para las otras siete cadenas. Hay que encontrar 101.

Búsqueda no estructurada. Clásicamente, $N - 1 = 7$ consultas en el peor caso y unas $N/2$ en media.

Paso 2. Superposición uniforme

$$|s\rangle = \frac{1}{\sqrt{8}}(1, 1, 1, 1, 1, 1, 1, 1), \quad \Pr(101) = \frac{1}{8}.$$

$$|s\rangle = H^{\otimes n} |0\rangle = \cos \theta |\alpha\rangle + \sin \theta |\beta\rangle, \text{ con } \sin \theta = \frac{1}{\sqrt{N}}.$$

Paso 3. Inversión de fase

$\frac{1}{\sqrt{8}}(1, 1, 1, 1, 1, -1, 1, 1)$: con el auxiliar en $|-\rangle$, el oráculo cambia el signo de 101.

$Z_f = I - 2|\mathbf{x}_0\rangle\langle\mathbf{x}_0|$ (retroceso de fase): **reflexión** respecto de $|\alpha\rangle$. El auxiliar se prepara en $|-\rangle$ una sola vez.

Paso 4. Inversión respecto de la media

Media $\frac{3}{4\sqrt{8}}$; nuevas amplitudes $\frac{1}{2\sqrt{8}}$ (siete veces) y $\frac{5}{2\sqrt{8}}$; $\Pr(101) = \frac{25}{32} \approx 0,78$.

$D = -I + 2A = 2|s\rangle\langle s| - I$: **reflexión** respecto de $|s\rangle$; unitaria porque $A^2 = A$. Se implementa como $H^{\otimes n}(2|0\rangle\langle 0| - I)H^{\otimes n}$.

Paso 5. Dos reflexiones, un giro

$$G|_P = \begin{pmatrix} \cos 2\theta & -\sin 2\theta \\ \sin 2\theta & \cos 2\theta \end{pmatrix}, \quad \theta \approx 20,7^\circ: \text{ cada iteración gira } 41,4^\circ \text{ hacia } |101\rangle.$$

Teorema: $G^k |s\rangle$ forma un ángulo $(2k + 1)\theta$ con $|\alpha\rangle$, y $\Pr(\mathbf{x}_0) = \sin^2((2k + 1)\theta)$.

Paso 6. Segunda iteración

Amplitudes $-\frac{1}{4\sqrt{8}}$ (siete veces) y $\frac{11}{4\sqrt{8}} \approx 0,972$; $\Pr(101) = \frac{121}{128} \approx 0,945 = \sin^2(5\theta)$.

$k = \lfloor \pi/(4\theta) \rfloor = 2$ iteraciones: $\Pr \geq 1 - \frac{1}{N}$.

Paso 7. No pasarse

Con $k = 3$: Pr = 0,330. Qiskit, 1000 disparos: 932 aciertos con $k = 2$ y 346 con $k = 3$.

El número de iteraciones es $\approx \frac{\pi}{4}\sqrt{N}$, **no** $\sqrt{N}$: el giro continúa y se aleja de la solución. Ventaja **cuadrática**, que es óptima.

Paso	Qué hicimos	Concepto
1	Buscar 101	Búsqueda no estructurada; cota clásica $\sim N$
2	$H^{\otimes 3} 000\rangle$	Estado uniforme, ángulo θ
3	Cambiar un signo	Inversión de fase = reflexión
4	$v \mapsto 2a - v$	Inversión respecto de la media = reflexión
5	$G = DZ_f$	Dos reflexiones = giro de 2θ
6	Segunda iteración	$\sin^2(5\theta) = \frac{121}{128}$
7	Tercera iteración	Pasarse de cocción; $k \approx \frac{\pi}{4}\sqrt{N}$

Bibliografía

- [1] N. S. Yanofsky y M. A. Mannucci. *Quantum Computing for Computer Scientists*. Cambridge University Press, 2008. Capítulo 6, secciones 6.1–6.4.
- [2] D. Deutsch. Quantum theory, the Church–Turing principle and the universal quantum computer. *Proceedings of the Royal Society of London A*, 400:97–117, 1985.
- [3] D. Deutsch y R. Jozsa. Rapid solution of problems by quantum computation. *Proceedings of the Royal Society of London A*, 439:553–558, 1992.
- [4] R. Cleve, A. Ekert, C. Macchiavello y M. Mosca. Quantum algorithms revisited. *Proceedings of the Royal Society of London A*, 454:339–354, 1998.
- [5] D. R. Simon. On the power of quantum computation. *SIAM Journal on Computing*, 26(5):1474–1483, 1997. (Versión de congreso: FOCS 1994.)
- [6] L. K. Grover. A fast quantum mechanical algorithm for database search. En *Proceedings of the 28th Annual ACM Symposium on Theory of Computing (STOC)*, 212–219, 1996.
- [7] M. Boyer, G. Brassard, P. Høyer y A. Tapp. Tight bounds on quantum searching. *Fortschritte der Physik*, 46:493–505, 1998.
- [8] C. H. Bennett, E. Bernstein, G. Brassard y U. Vazirani. Strengths and weaknesses of quantum computing. *SIAM Journal on Computing*, 26(5):1510–1523, 1997.
- [9] M. A. Nielsen e I. L. Chuang. *Quantum Computation and Quantum Information*. Cambridge University Press, edición del 10.^o aniversario, 2010.
- [10] J. Watrous. *Understanding Quantum Information and Computation*. IBM Quantum Learning. <https://quantum.cloud.ibm.com/learning>
- [11] V. Havlíček, A. D. Córcoles, K. Temme, A. W. Harrow, A. Kandala, J. M. Chow y J. M. Gambetta. Supervised learning with quantum-enhanced feature spaces. *Nature*, 567:209–212, 2019.
- [12] S. Thanasilp, S. Wang, M. Cerezo y Z. Holmes. Exponential concentration in quantum kernel methods. *Nature Communications*, 15:5200, 2024.